

Advanced Machine learning Mastering Course

Introduced by

George Samuel

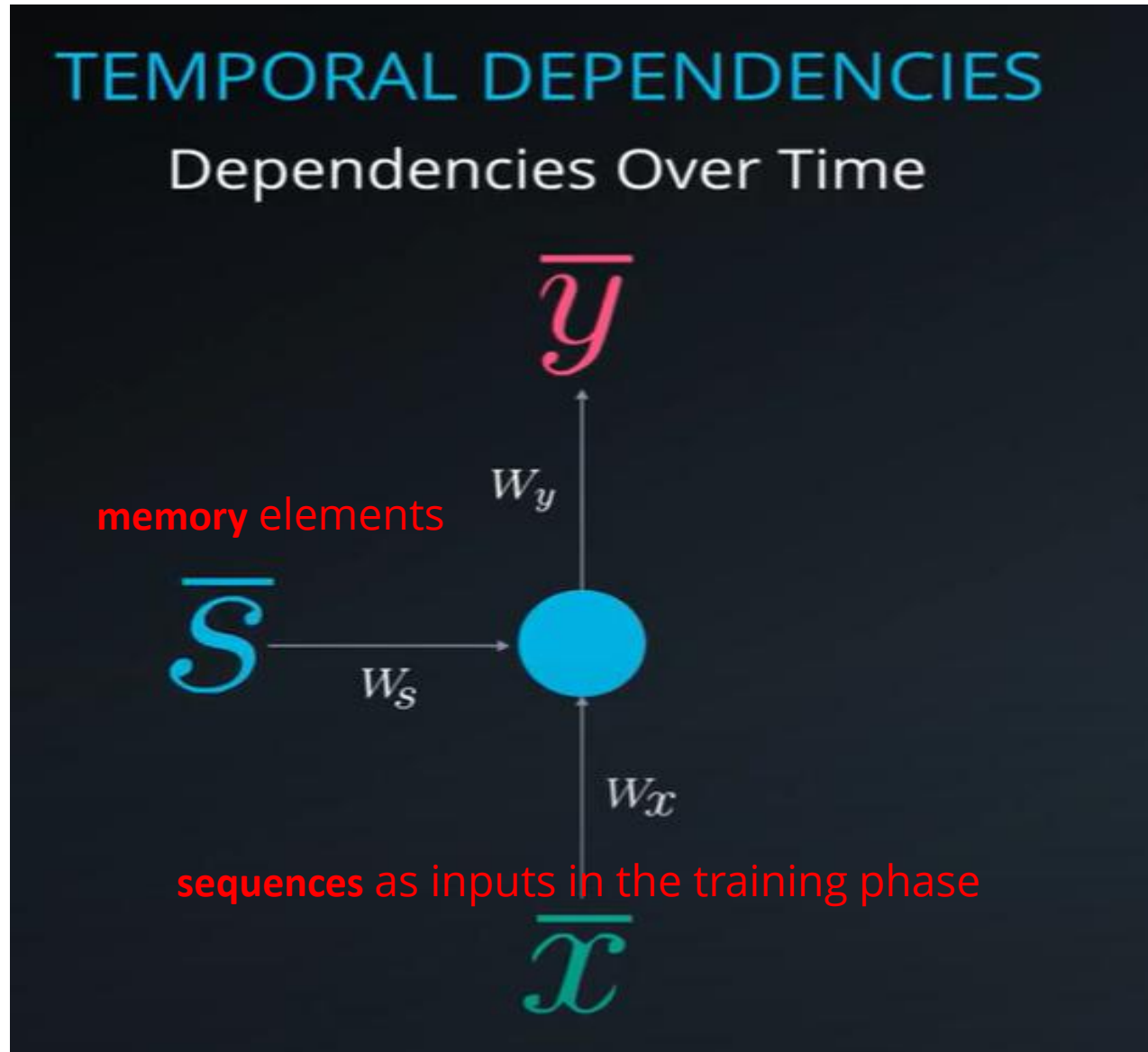
Recurrent Neural Network

GRU-LSTM

Innovisionray.com

2024

Recurrent Neural Network

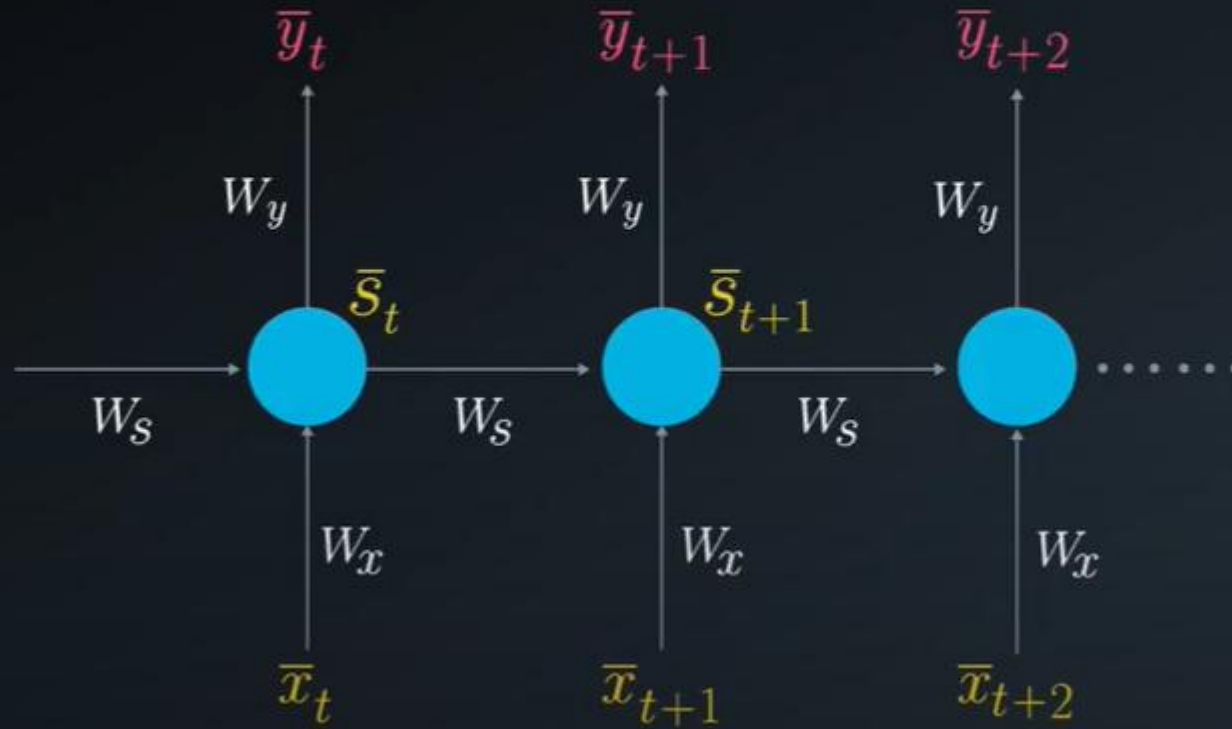


Recurrent Neural Network

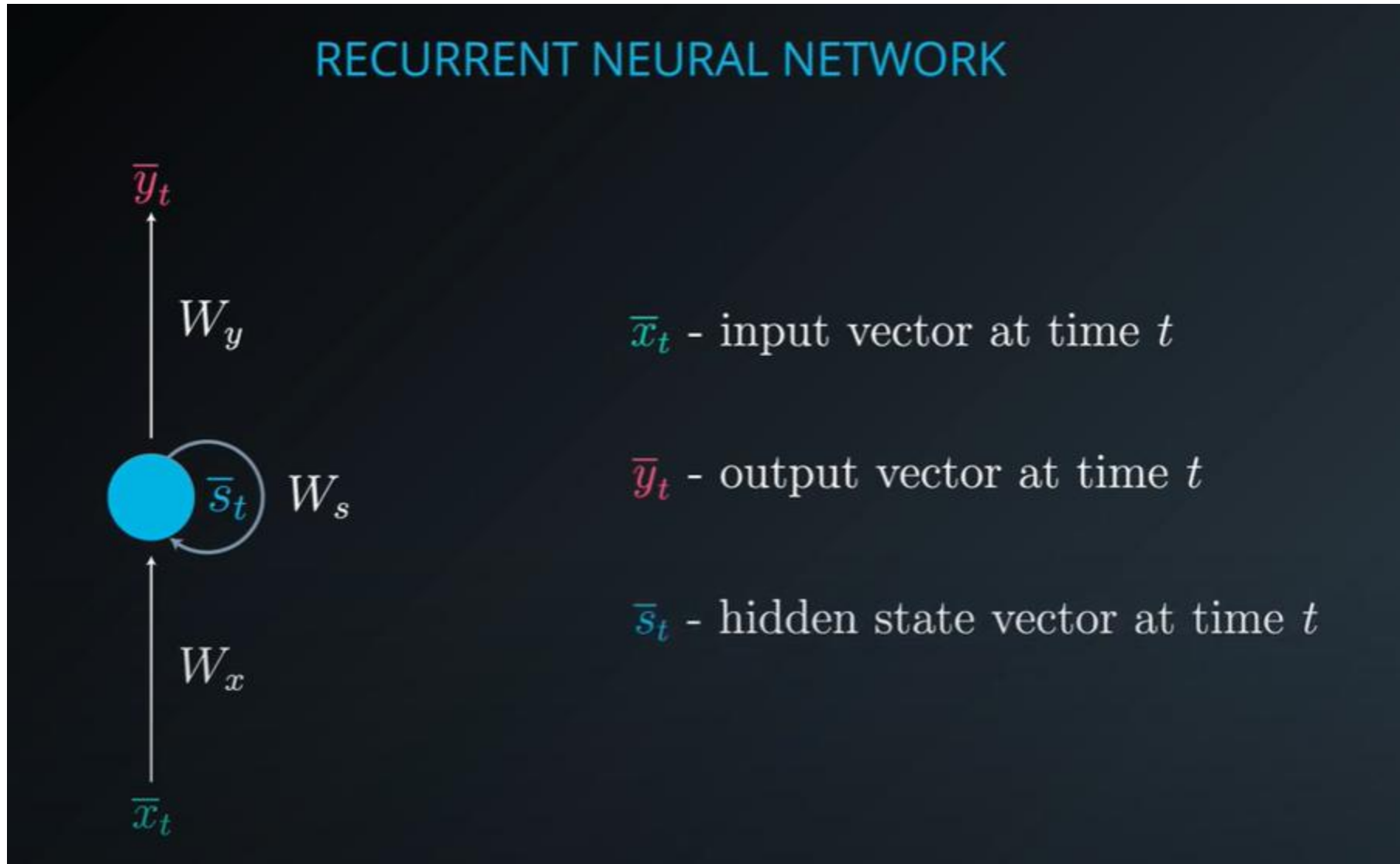
- Sentiment Analysis
- Speech Recognition
- Time Series Prediction
- Natural Language Processing
- Gesture Recognition

Recurrent Neural Network

RECURRENT NEURAL NETWORKS

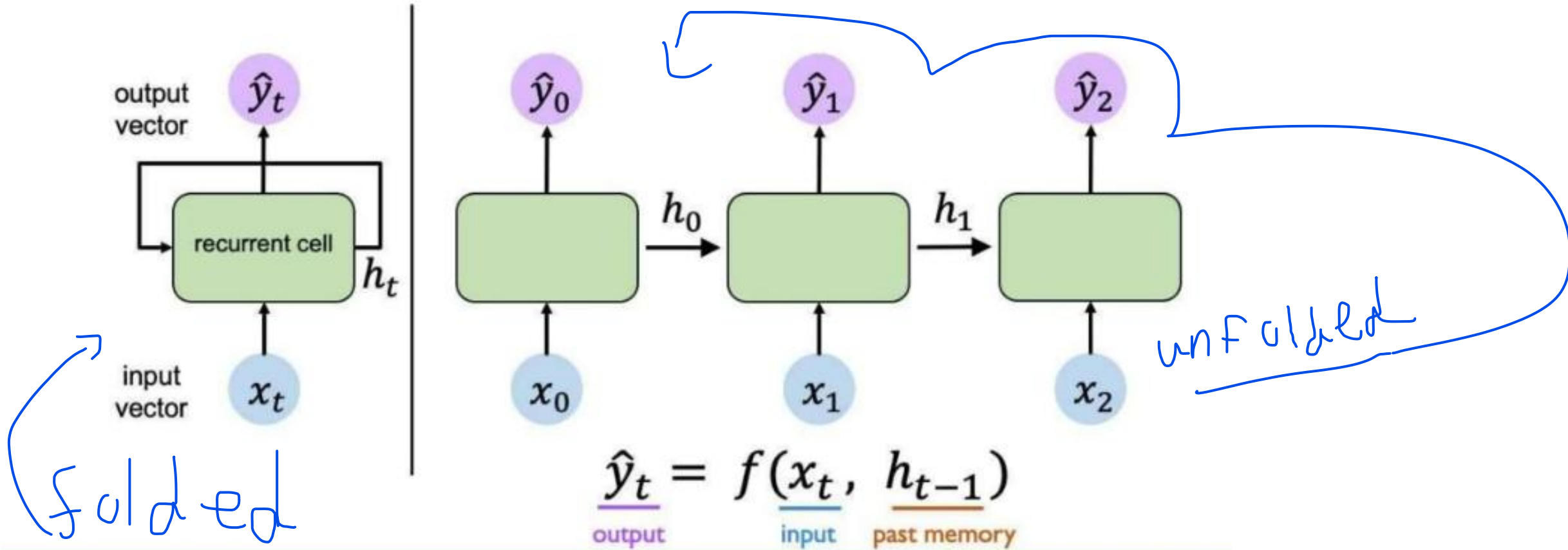


Recurrent Neural Network



Recurrent Neural Network

Neurons with Recurrence



Recurrent Neural Network

RECURRENT NEURAL NETWORK (RNN)

Recurrent: perform same task for every element of a sequence

Output: depend on:

previous computations as well as
new inputs

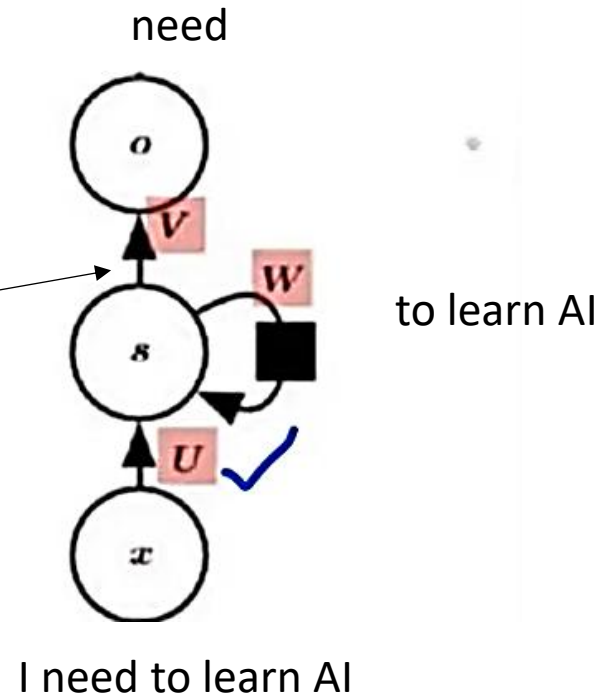
RNNs have a "memory" of past !

current state = $f(\text{previous state}, \text{input})$

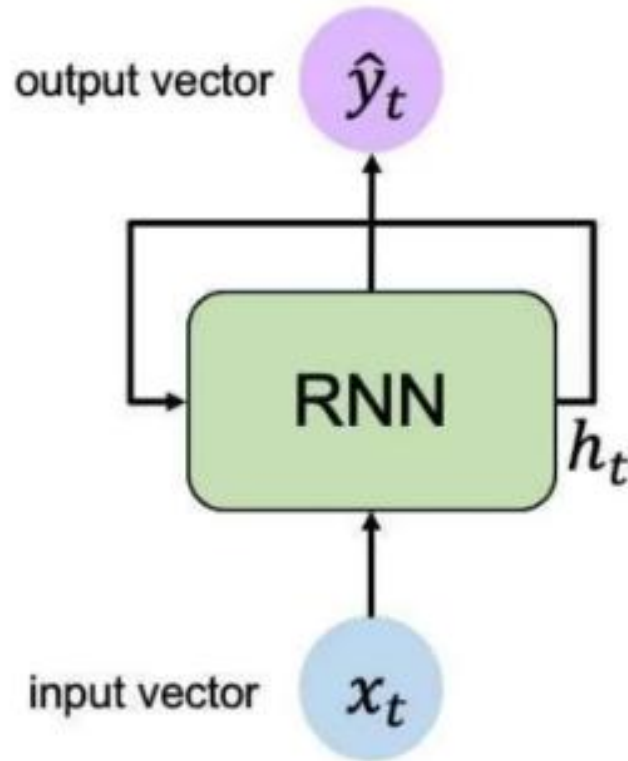
$$s_t = f(s_{t-1}, x_t) = \tanh(Ws_{t-1} + Ux_t)$$

$$o_t = Vs_t$$

Output of the previous state



Recurrent Neural Networks (RNNs)



Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

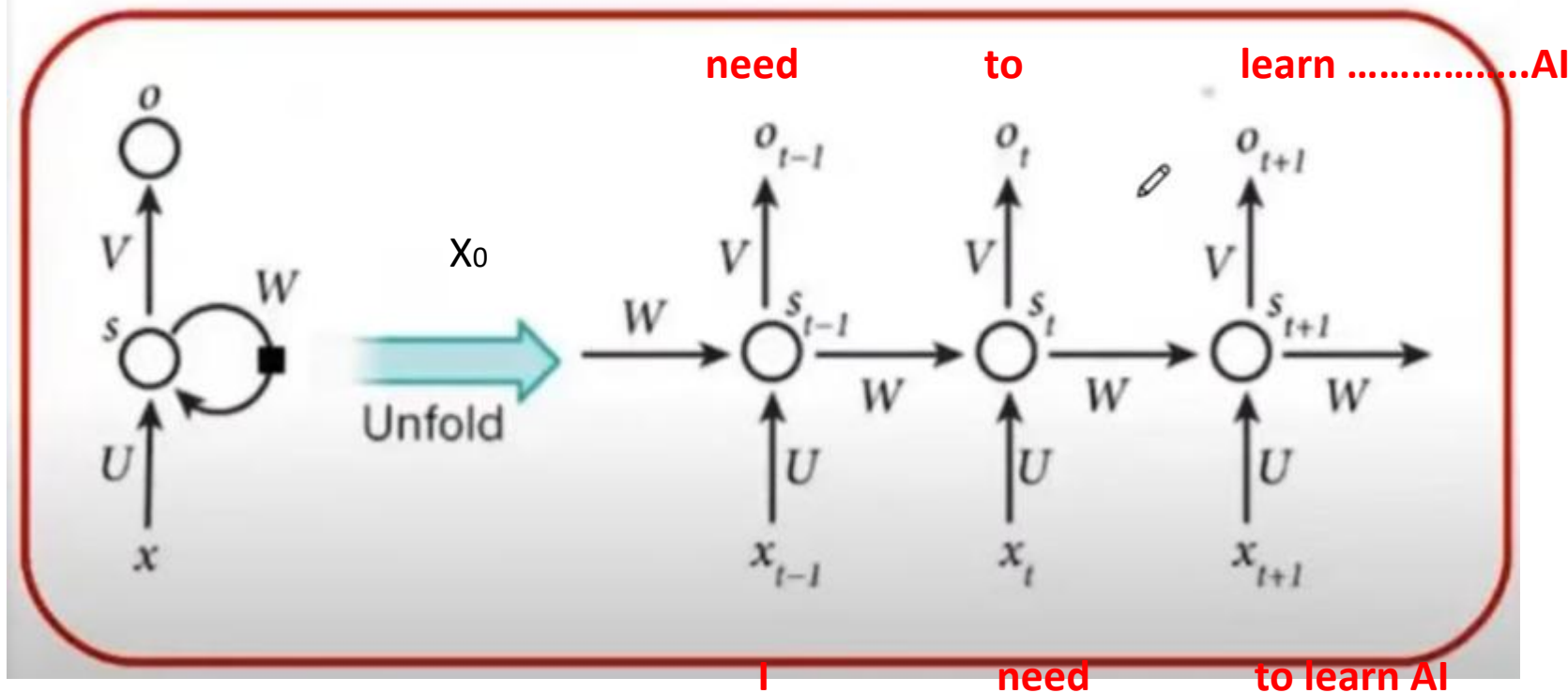
cell state function with weights W input old state

Note: the same function and set of parameters are used at every time step

RNNs have a **state**, h_t , that is updated **at each time step** as a sequence is processed

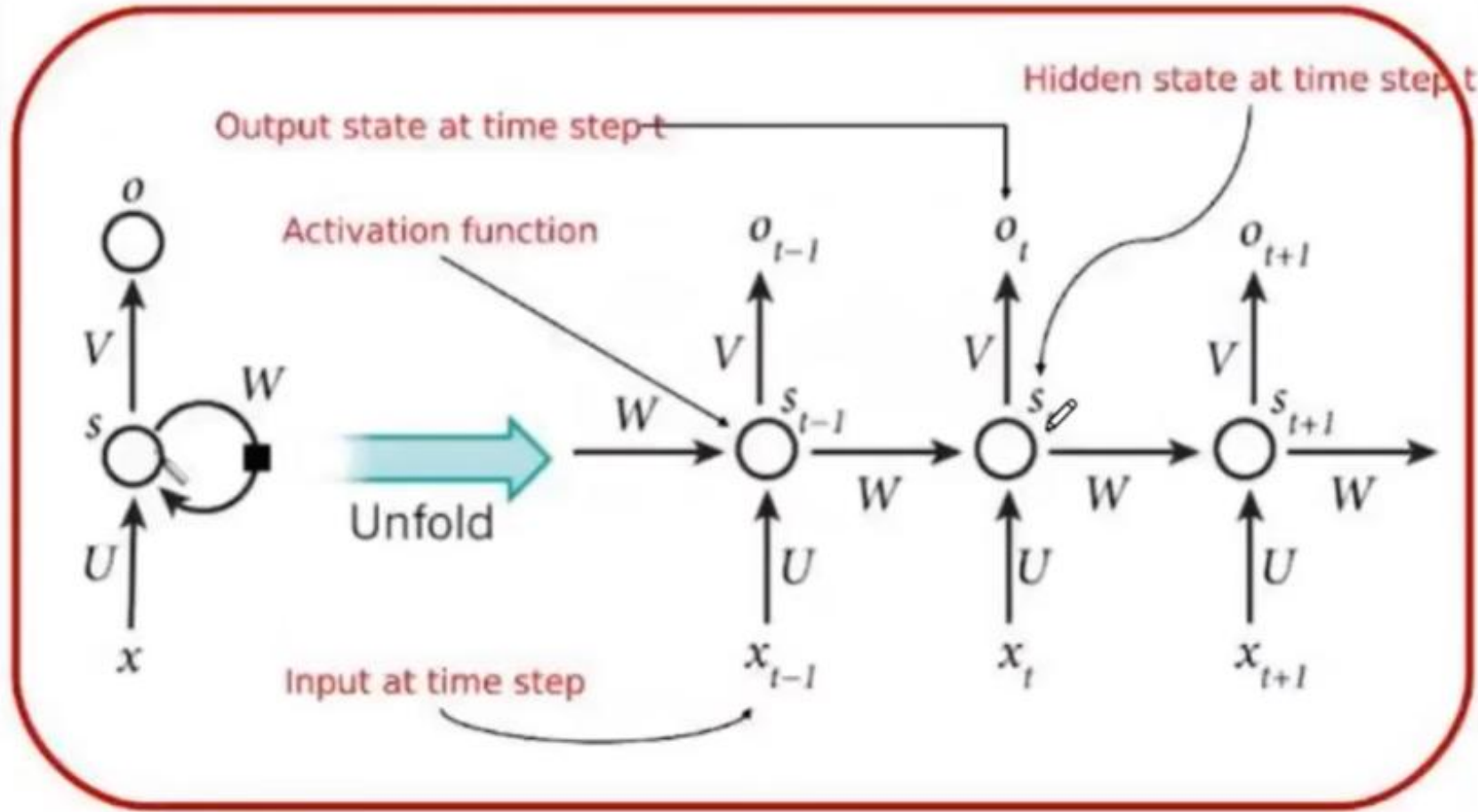
Recurrent Neural Network

RNN (FOLDED - UNFOLDED)

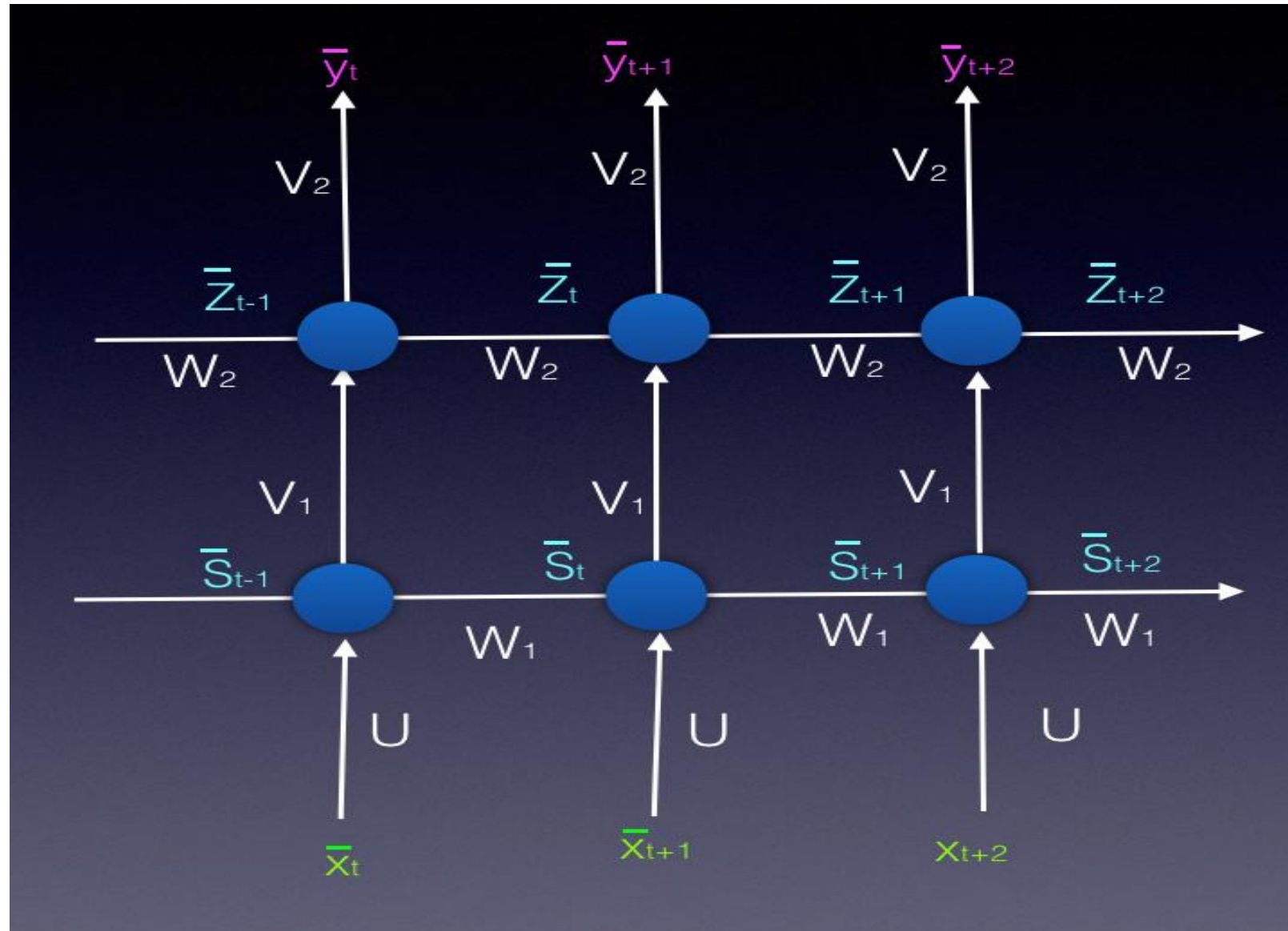
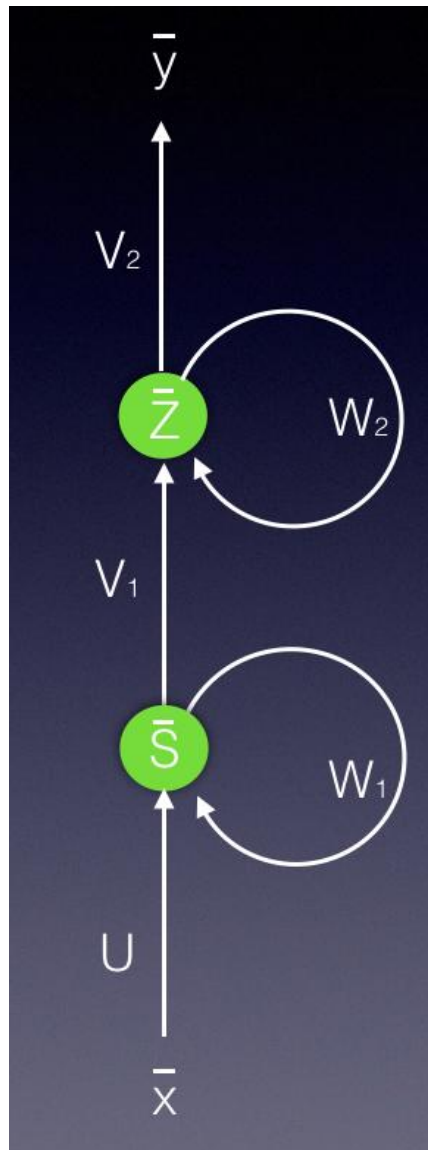


Recurrent Neural Network

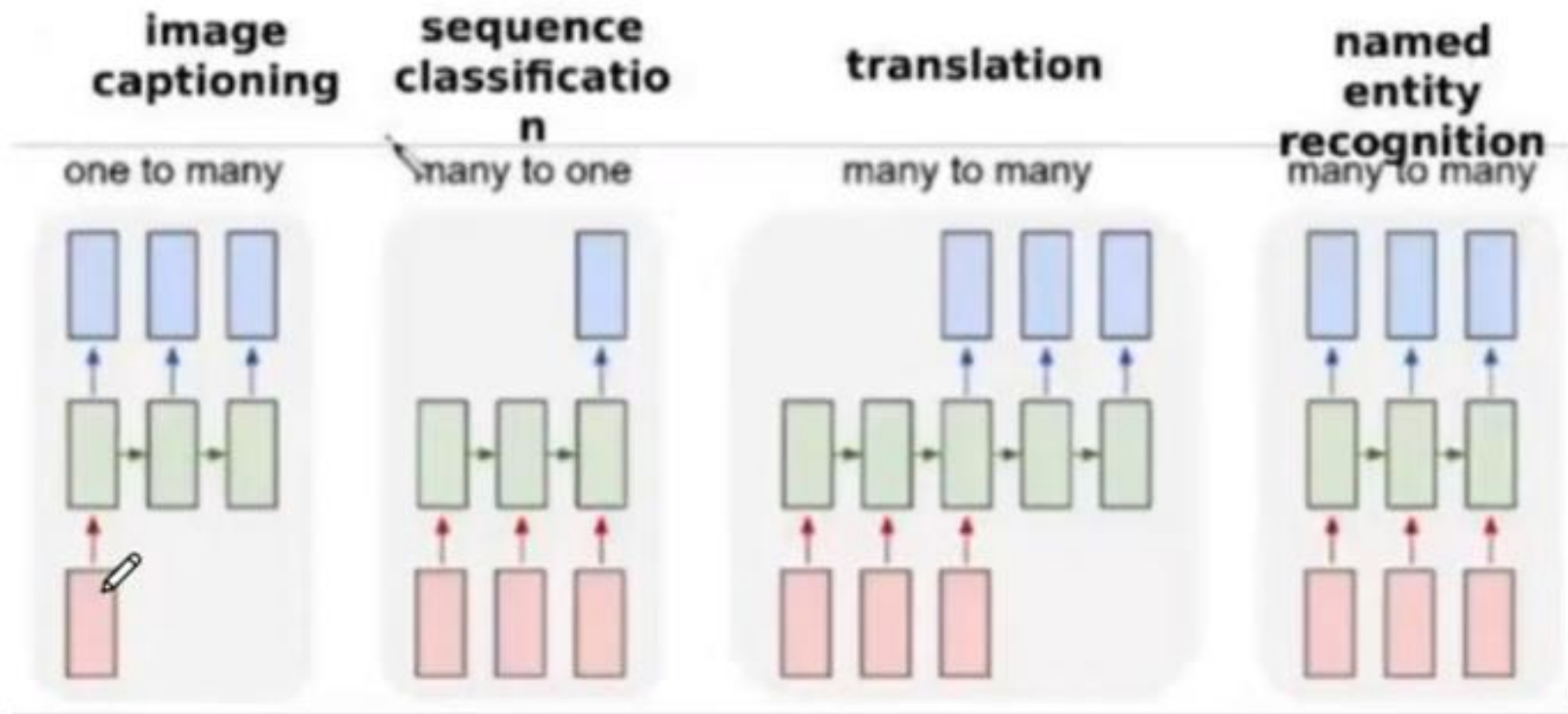
RNN (FOLDED - UNFOLDED)



Recurrent Neural Network



Examples of Sequence Application of RNN - LSTM



CHARACTER-LEVEL LANGUAGE MODEL

- Vocabulary:
 - Only 4 characters [h, e, l, o]
- Training sequence “hello”
 1. Apply input with 4×1 vocabulary vector sequentially
 2. Calculate hidden states: $s_t = \tanh(Ws_{t-1} + Ux_t)$
 3. Calculate output: $o_t = Vs_t$
 4. Compare with next character & calculate loss
 5. Back-propagate errors and updates weights(U,V,W)

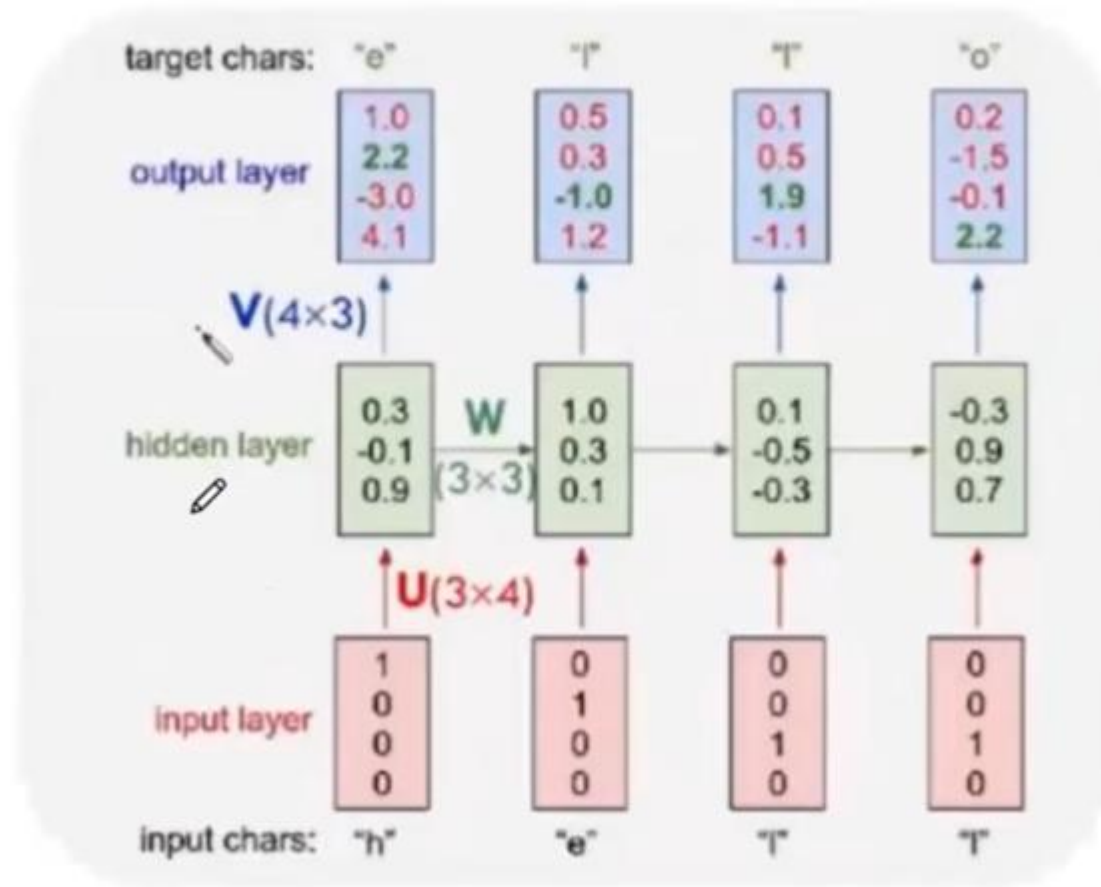
One-HOT Vectors input for Word Sequence

- A vector with all zeros except for the active dimension
- 12 words in a sequence → 12 One-hot vectors
- After the one-hot vectors apply an embedding (Word2Vec, GloVE)

<u>Vocabulary</u>			<u>One-hot vectors</u>					
I	I	1	I	0	I	0	I	0
am	am	0	am	1	am	0	am	0
Bond	Bond	0	Bond	0	Bond	1	Bond	0
James	James	0	James	0	James	0	James	1
tired	tired	0	tired	0	tired	0	tired	0
,	,	0	,	0	,	0	,	0
McQuire	McQuire	0	McQuire	0	McQuire	0	McQuire	0
I	I	0	I	0	I	0	I	0

Recurrent Neural Network

CHARACTER-LEVEL LANGUAGE MODEL



CHARACTER-LEVEL LANGUAGE MODEL

■ Training

- Softmax classifier on output vector
- Use a cross-entropy loss function
- Back Propagation Through Time(**BPTT**)

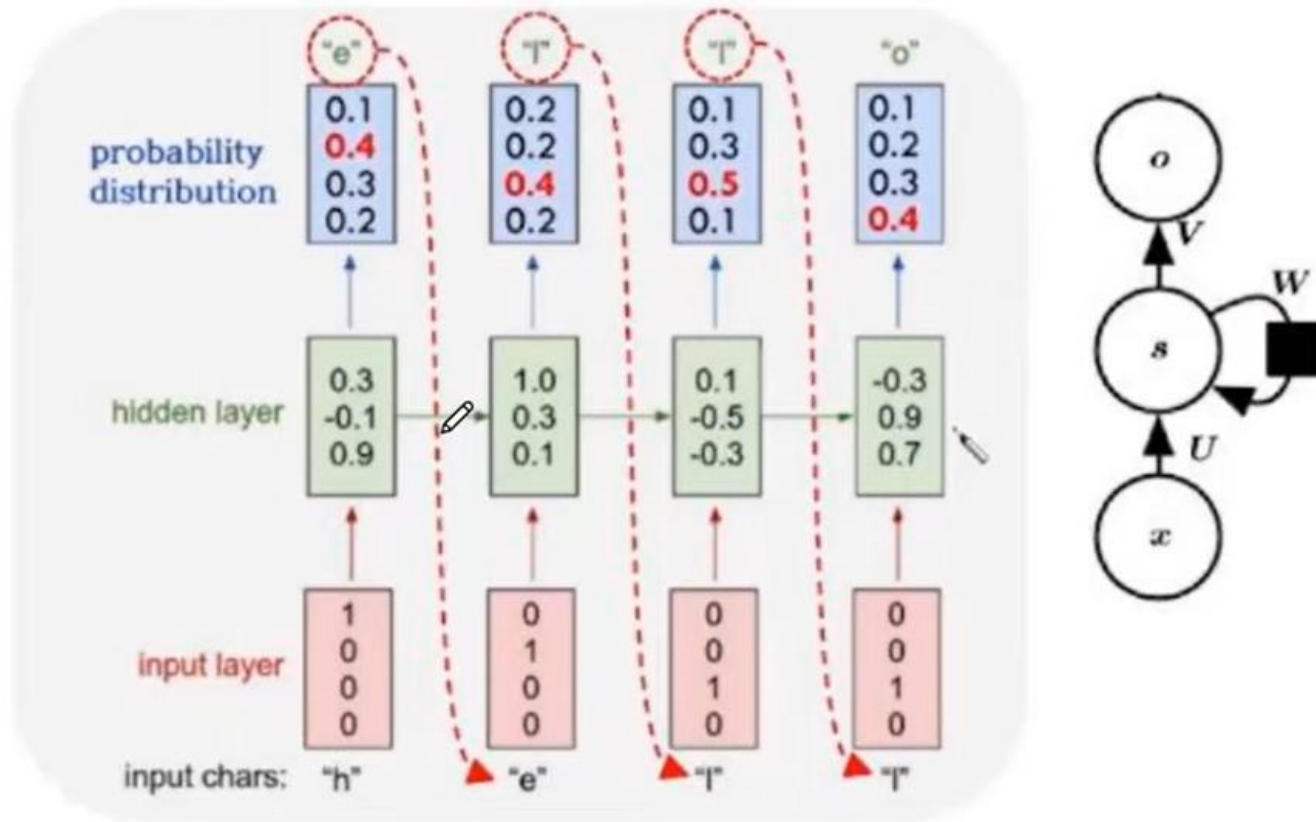


■ Test

1. Feed a character into the RNN
2. Get a distribution over output
3. Find character with highest probability
4. Feed the highest into RNN input

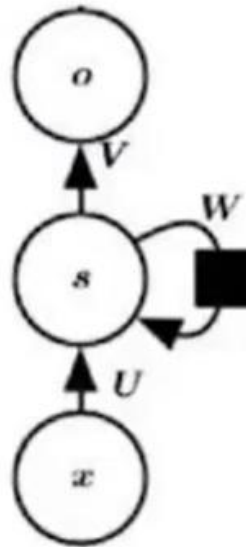
Recurrent Neural Network

CHARACTER-LEVEL LANGUAGE MODEL (Generative Model)



Simple and Real RNN (Number of Parameters)

	Simple RNN	Real RNN
Vocabulary	4 	80
Hidden Layers	1	3
Hidden nodes	3	100
U	3×4	100×80
W	3×3	$100 \times 100 \times 3$
W_{hh}	-	$100 \times 100 \times 2$
V	4×3	80×100
Total parameters	33	66,000

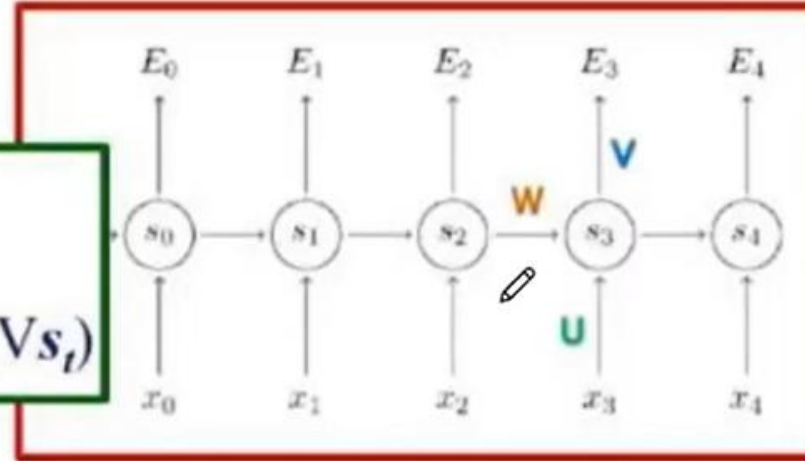


Recurrent Neural Network

BACKPROPAGATION THROUGH TIME (BPTT)

Forward calculation

- $s_t = \tanh(Ws_{t-1} + Ux_t)$
- $\hat{y}_t = \text{softmax}(o_t) = \text{softmax}(Vs_t)$



With cross entropy loss

- $E_t(y_t, \hat{y}_t) = -y_t \log \hat{y}_t$
- $E(y, \hat{y}) = \sum_t E_t(y_t, \hat{y}_t) = -\sum_t y_t \log \hat{y}_t$

y_t : correct output

$\hat{y}_t$: calculated output of the network

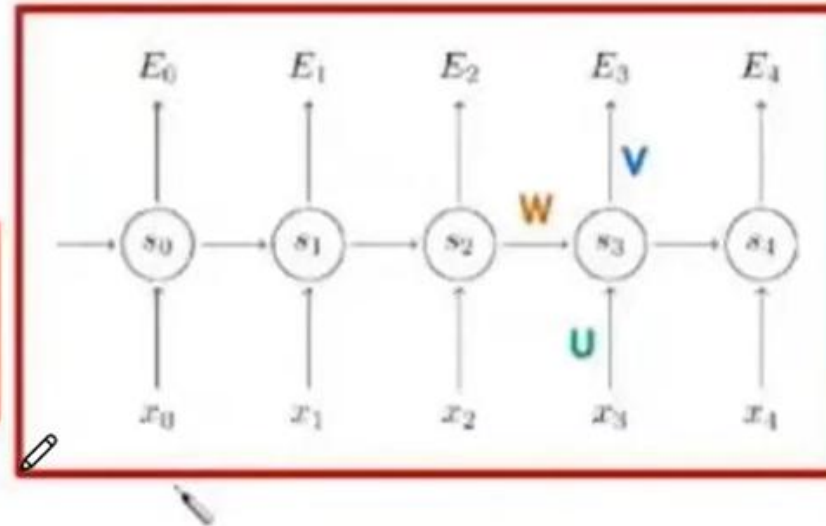
- Training with a full sentence as one example

Recurrent Neural Network

BACKPROPAGATION THROUGH TIME (BPTT)

Calculate gradients of error
with respect to: **U**, **V**, **W**.

$$\begin{aligned}s_3 &= \tanh(Ws_2 + Ux_3) \\ s_2 &= \tanh(Ws_1 + Ux_2) \\ s_1 &= \tanh(Ws_0 + Ux_1)\end{aligned}$$



Sum up the gradients at each time step

$$\frac{\partial E}{\partial U} = \sum_t \frac{\partial E_t}{\partial U}, \quad \frac{\partial E}{\partial V} = \sum_t \frac{\partial E_t}{\partial V}, \quad \frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$

Recurrent Neural Network

BACKPROPAGATION THROUGH TIME (BPTT)

Calculate Gradients by chain Rule

$$\frac{\partial E_3}{\partial V} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial o_3} \cdot \frac{\partial o_3}{\partial V}$$

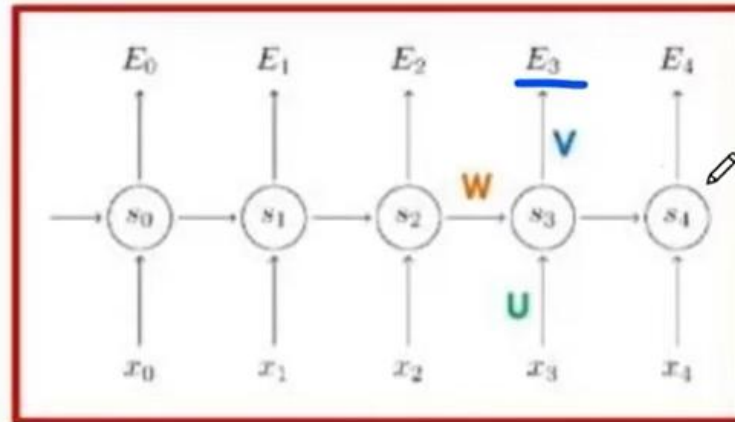
Remember:

$$E_t(y_t, \hat{y}_t) = -y_t \log \hat{y}_t$$

$$\hat{y}_t = \text{softmax}(o_t) = \text{softmax}(Vs_t)$$

$$o_t = Vs_t$$

$$s_t = \tanh(Ws_{t-1} + Ux_t)$$



$$\begin{aligned} s_3 &= \tanh(Ws_2 + Ux_3) \\ s_2 &= \tanh(Ws_1 + Ux_2) \\ s_1 &= \tanh(Ws_0 + Ux_1) \end{aligned}$$

Softmax f'

$$\hat{y}_k = \frac{e^{a_k}}{\sum_{k'=1}^K e^{a_{k'}}} \text{ for } k = 1 \text{ to } k = K.$$

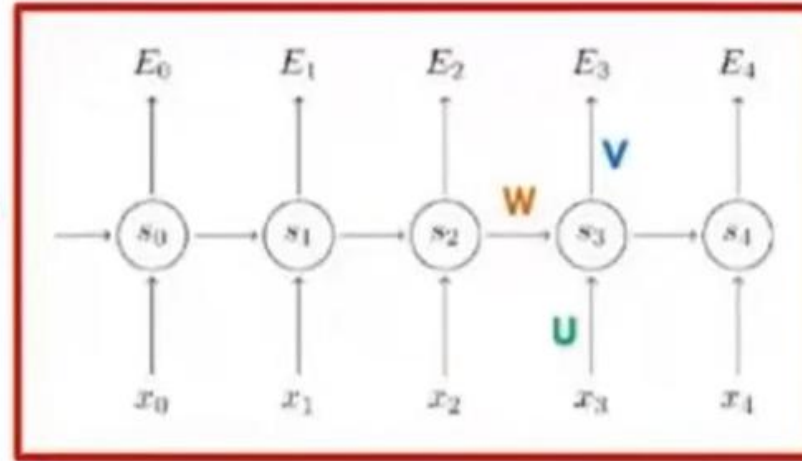
Recurrent Neural Network

BACKPROPAGATION THROUGH TIME (BPTT)

$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial W} \quad \frac{\partial s_3}{\partial W} ?$$

s_1 and s_2 Depends on W and U too

$$\frac{\partial s_3}{\partial W} \rightarrow \sum_k \frac{\partial s_3}{\partial s_k} \cdot \frac{\partial s_k}{\partial W}$$



$$\frac{\partial E_3}{\partial W} = \sum_k \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_k} \cdot \frac{\partial s_k}{\partial W}$$

$$\begin{aligned} s_3 &= \tanh(Ws_2 + Ux_3) \\ s_2 &= \tanh(Ws_1 + Ux_2) \\ s_1 &= \tanh(Ws_0 + Ux_1) \end{aligned}$$

$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial W} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_2} \cdot \frac{\partial s_2}{\partial W} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_1} \cdot \frac{\partial s_1}{\partial W} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_0} \cdot \frac{\partial s_0}{\partial W}$$

Recurrent Neural Network

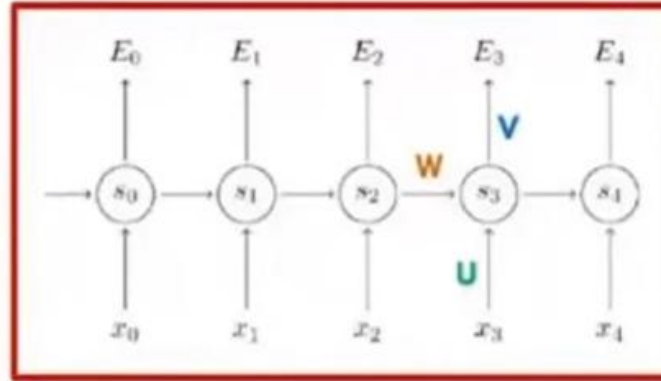
BACKPROPAGATION THROUGH TIME (BPTT)

$$\frac{\partial E_3}{\partial U} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial U}$$

$$\frac{\partial s_3}{\partial U} ?$$

s_1 and s_2 Depends on W and U too

$$\frac{\partial s_3}{\partial U} \rightarrow \sum_k \frac{\partial s_3}{\partial s_k} \cdot \frac{\partial s_k}{\partial U}$$



$$\frac{\partial E_3}{\partial U} = \sum_k \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_k} \cdot \frac{\partial s_k}{\partial U}$$

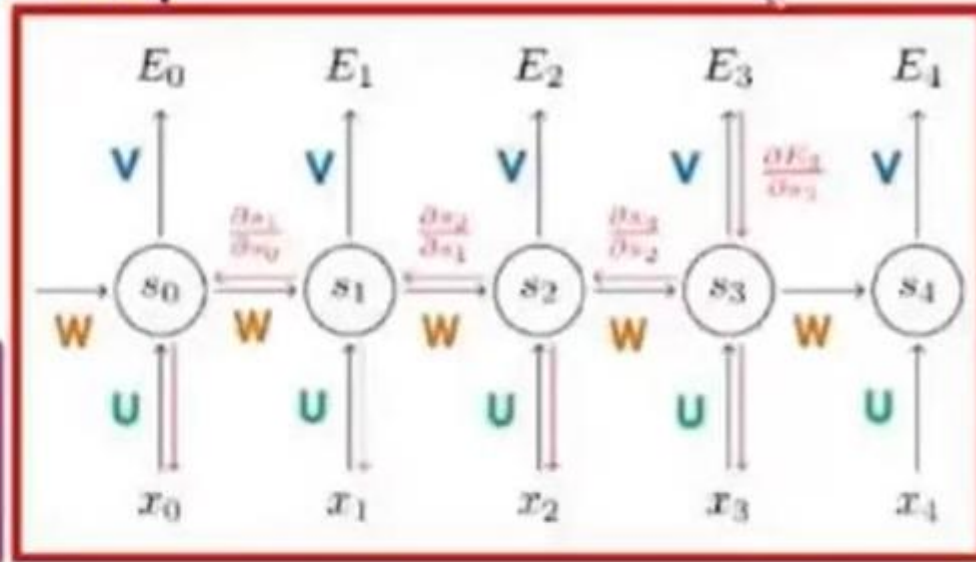
$$\begin{aligned} s_3 &= \tanh(Ws_2 + Ux_3) \\ s_2 &= \tanh(Ws_1 + Ux_2) \\ s_1 &= \tanh(Ws_0 + Ux_1) \end{aligned}$$

$$\frac{\partial E_3}{\partial U} = \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial U} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_2} \cdot \frac{\partial s_2}{\partial U} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_1} \cdot \frac{\partial s_1}{\partial U} + \frac{\partial E_3}{\partial \hat{y}_3} \cdot \frac{\partial \hat{y}_3}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_0} \cdot \frac{\partial s_0}{\partial U}$$

Recurrent Neural Network

Sum up the gradients at each time step

$$\frac{\partial E}{\partial V} = \sum_t \frac{\partial E_t}{\partial V} = \sum_t \frac{\partial E_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial o_t} \cdot \frac{\partial o_t}{\partial V}$$



$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W} = \sum_t \sum_k \frac{\partial E_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_k} \cdot \frac{\partial s_k}{\partial W}$$

*Propagation through time
(RNN)*

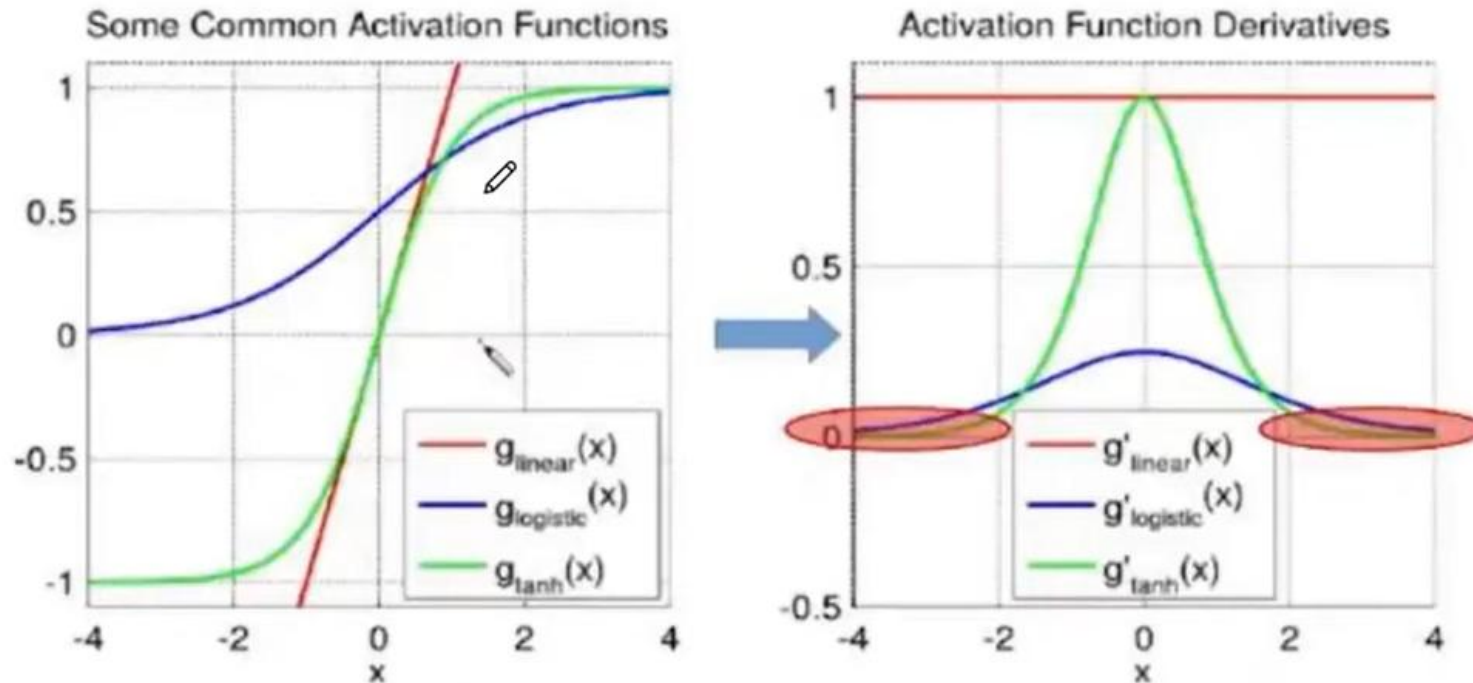
=

*Propagation through layers
(FNN)*

$$\frac{\partial E}{\partial U} = \sum_t \frac{\partial E_t}{\partial U} = \sum_t \sum_k \frac{\partial E_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_k} \cdot \frac{\partial s_k}{\partial U}$$

Gradients of Some Common Activation functions

Vanishing gradients



Recurrent Neural Network

VANISHING GRADIENT PROBLEM

Error Gradients pass through nonlinearity every step

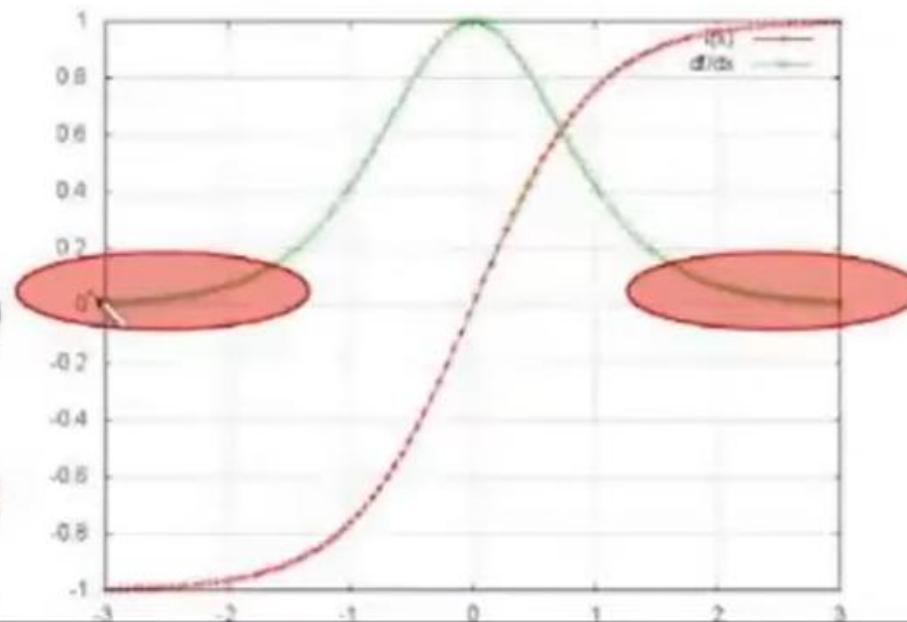
Saturation at both ends $\Rightarrow$ zero gradient

Vanishing completely after a few time steps.

$$\frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial}{\partial s_{t-1}} \tanh(Ws_{t-1} + Ux_t)$$

Tanh Derivative ranges from 0 to 1

tanh and derivative



VANISHING GRADIENT PROBLEM

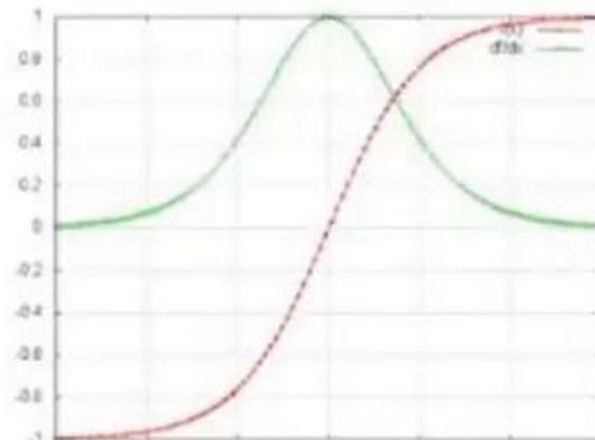
$$\frac{\partial \mathcal{L}}{\partial c_t} = \frac{\partial \mathcal{L}}{\partial c_T} \cdot \underbrace{\frac{\partial c_T}{\partial c_{T-1}}}_{\rho < 1} \cdot \underbrace{\frac{\partial c_{T-1}}{\partial c_{T-2}}}_{\rho < 1} \cdot \dots \cdot \underbrace{\frac{\partial c_{t+1}}{\partial c_t}}_{\rho < 1} \left\} \frac{\partial \mathcal{L}}{\partial c_t} \ll 1 \Rightarrow \text{Vanishing gradient}$$

$$\frac{\partial \mathcal{L}}{\partial c_t} = \frac{\partial \mathcal{L}}{\partial c_T} \cdot \underbrace{\frac{\partial c_T}{\partial c_{T-1}}}_{\rho > 1} \cdot \underbrace{\frac{\partial c_{T-1}}{\partial c_{T-2}}}_{\rho > 1} \cdot \dots \cdot \underbrace{\frac{\partial c_{t+1}}{\partial c_t}}_{\rho > 1} \left\} \frac{\partial \mathcal{L}}{\partial c_t} \gg 1 \Rightarrow \text{Exploding gradient}$$

$$\frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial}{\partial s_{t-1}} \tanh(Ws_{t-1} + Ux_t)$$

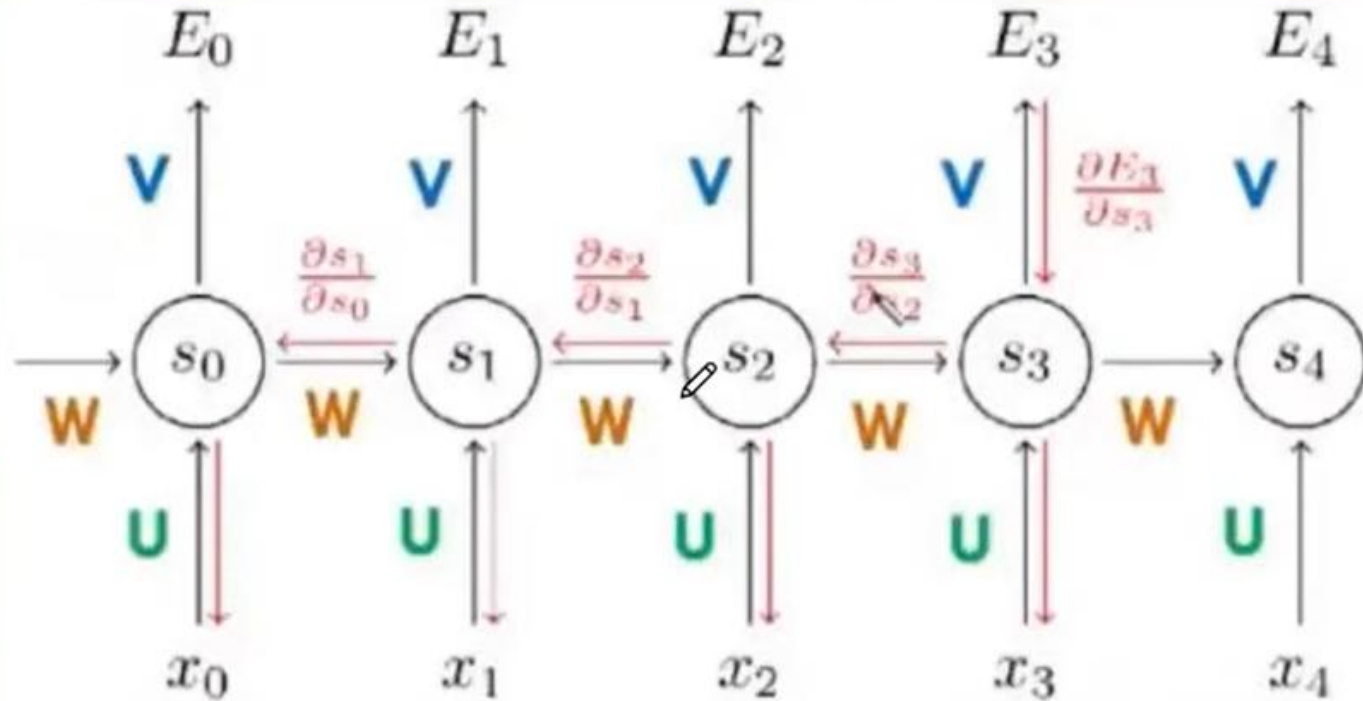
Tanh Derivative ranges from 0 to 1

tanh and derivative



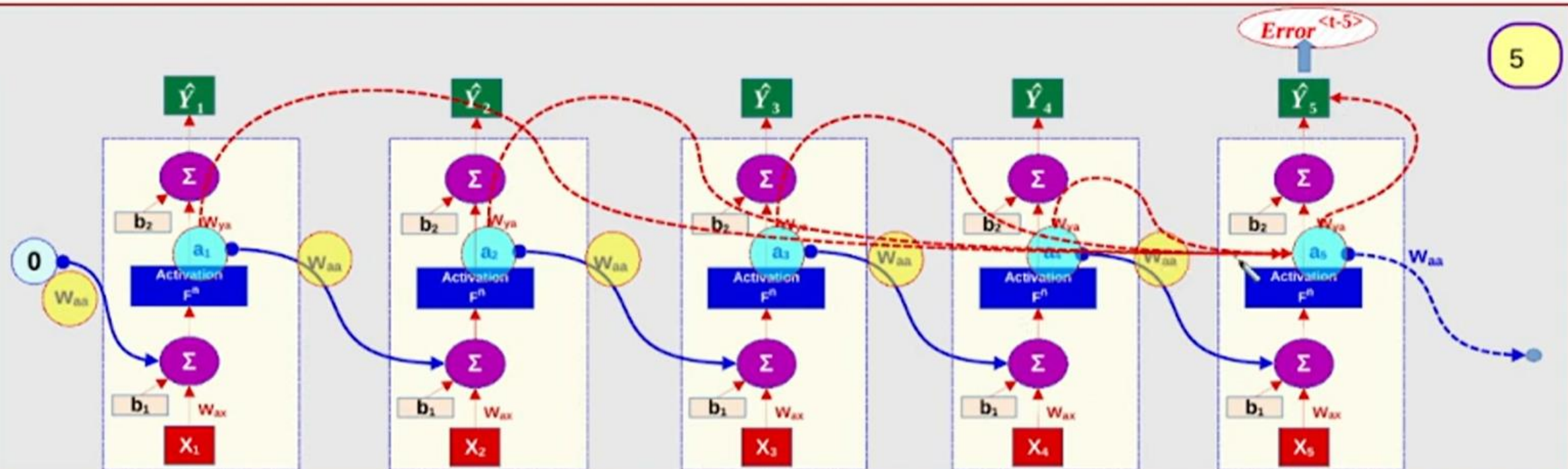
Recurrent Neural Network

VANISHING GRADIENT PROBLEM



$$\frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial}{\partial s_{t-1}} \tanh(Ws_{t-1} + Ux_t) \quad \text{Tanh Derivative ranges from 0 to 1}$$

From Recurrent Neural Network (RNN) to Lone Short Term Memory (LSTM)



$$\frac{\partial \text{Error}_{(\hat{y}_5)}}{\partial w_{aa}} = \frac{\partial \text{Error}_{(\hat{y}_5)}}{\partial \hat{y}_5} * \frac{\partial \hat{y}_5}{\partial w_{aa}} =$$

Remember: $w_{aa}^{\text{updated}} = w_{aa}^{\text{old}} - \eta * \frac{\partial \text{Error}_{(\text{Total})}}{\partial w_{aa}}$

$$= \frac{\partial \text{Error}_{(\hat{y}_5)}}{\partial \hat{y}_5} \left[\frac{\partial \hat{y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{y}_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{y}_5}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{y}_5}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \frac{\partial \hat{y}_5}{\partial a_1} * \frac{\partial a_1}{\partial w_{aa}} \right]$$

$$= \frac{\partial \text{Error}_{(\hat{y}_5)}}{\partial \hat{y}_5} \left[\frac{\partial \hat{y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \dots \right]$$

Gradient of Error at Y_5 due to W_{aa}

7



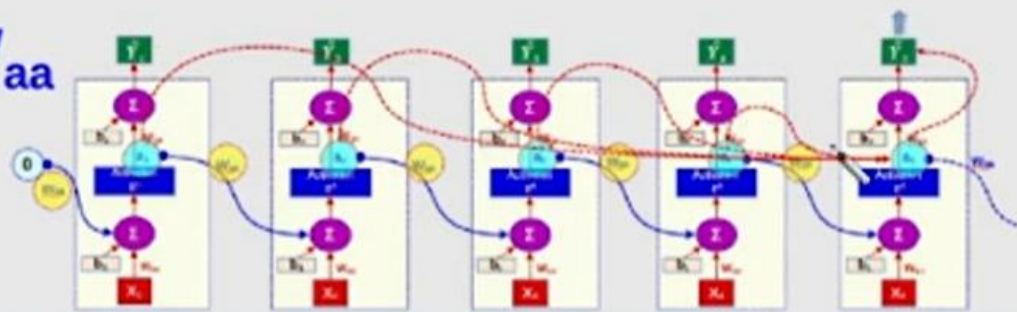
$$\begin{aligned} \frac{\partial \text{Error}_{(Y_5)}}{\partial W_{aa}} &= \frac{\partial \text{Error}_{(Y_5)}}{\partial \hat{Y}_5} * \frac{\partial \hat{Y}_5}{\partial W_{aa}} = \\ &= \frac{\partial \text{Error}_{(Y_5)}}{\partial \hat{Y}_5} \left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial W_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_4} * \frac{\partial a_4}{\partial W_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_3} * \frac{\partial a_3}{\partial W_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_2} * \frac{\partial a_2}{\partial W_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_1} * \frac{\partial a_1}{\partial W_{aa}} \right] \\ &= \frac{\partial \text{Error}_{(Y_5)}}{\partial \hat{Y}_5} \left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial W_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial W_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial W_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial a_2} * \frac{\partial a_2}{\partial W_{aa}} + \dots \right] \end{aligned}$$

$$\frac{\partial \text{Error}_{(Y_5)}}{\partial W_{aa}} = \frac{\partial \text{Error}_{(Y_5)}}{\partial Y_5} \sum_{k=1}^5 \left[\frac{\partial \hat{Y}_5}{\partial a_k} * \frac{\partial a_k}{\partial W_{aa}} \right] = \sum_{k=1}^5 \frac{\partial \text{Error}_{(Y_5)}}{\partial Y_5} * \frac{\partial \hat{Y}_5}{\partial a_k} * \frac{\partial a_k}{\partial W_{aa}}$$

Move inside the summation

Gradient of Error at Y_5 due to W_{aa}

9



$$\begin{aligned} \frac{\partial \text{Error}_{(\hat{Y}_5)}}{\partial w_{aa}} &= \frac{\partial \text{Error}_{(\hat{Y}_5)}}{\partial \hat{Y}_5} * \frac{\partial \hat{Y}_5}{\partial w_{aa}} = \\ &= \frac{\partial \text{Error}_{(\hat{Y}_5)}}{\partial \hat{Y}_5} \left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_1} * \frac{\partial a_1}{\partial w_{aa}} \right] \\ &= \frac{\partial \text{Error}_{(\hat{Y}_5)}}{\partial \hat{Y}_5} \left[\frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial w_{aa}} + \frac{\partial \hat{Y}_5}{\partial a_5} * \frac{\partial a_5}{\partial a_4} * \frac{\partial a_4}{\partial a_3} * \frac{\partial a_3}{\partial a_2} * \frac{\partial a_2}{\partial w_{aa}} + \dots \right] \end{aligned}$$

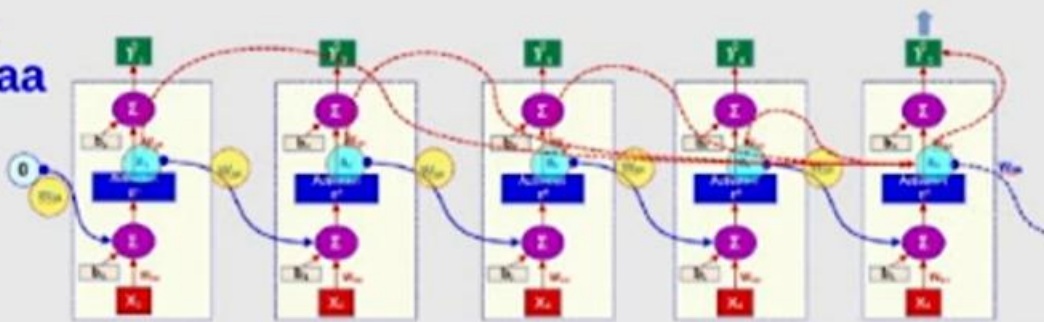
$$\frac{\partial \text{Error}_{(\hat{Y}_5)}}{\partial w_{aa}} = \frac{\partial \text{Error}_{(\hat{Y}_5)}}{\partial Y_5} \sum_{k=1}^5 \left[\frac{\partial \hat{Y}_5}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}} \right] = \sum_{k=1}^5 \frac{\partial \text{Error}_{(\hat{Y}_5)}}{\partial Y_5} * \frac{\partial \hat{Y}_5}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}} = \sum_{k=1}^5 \frac{\partial \text{Error}_{(\hat{Y}_5)}}{\partial Y_5} * \frac{\partial a_k}{\partial w_{aa}} * \frac{\partial \hat{Y}_5}{\partial a_5} * \prod_{j=k}^{5-1} \frac{\partial a_{j+1}}{\partial a_j}$$

In General Error_t (due to Y_t ONLY) $\Rightarrow$ $\frac{\partial \text{Error}_t}{\partial w_{aa}} = \frac{\partial \text{Error}_t}{\partial Y_t} \sum_{k=1}^t \left[\frac{\partial \hat{Y}_t}{\partial a_k} * \frac{\partial a_k}{\partial w_{aa}} \right] = \sum_{k=1}^t \frac{\partial \text{Error}_t}{\partial Y_t} * \frac{\partial a_k}{\partial w_{aa}} * \frac{\partial \hat{Y}_t}{\partial a_t} * \prod_{j=k}^{t-1} \frac{\partial a_{j+1}}{\partial a_j}$

t=5 in the previous derivation

Gradient of Total Error due to W_{aa}

10



In General: **Error_t** (due to Y_t ONLY)

$$\frac{\partial \text{Error}_t}{\partial W_{aa}} = \frac{\partial \text{Error}_t}{\partial Y_t} \sum_{k=1}^t \left[\frac{\partial \hat{Y}_t}{\partial a_k} * \frac{\partial a_k}{\partial W_{aa}} \right] = \sum_{k=1}^t \frac{\partial \text{Error}_t}{\partial Y_t} * \frac{\partial a_k}{\partial W_{aa}} * \frac{\partial \hat{Y}_t}{\partial a_t} * \prod_{j=k}^{t-1} \frac{\partial a_{j+1}}{\partial a_j}$$

Total Error (at all time steps) due to changes in W_{aa}

$$\frac{\partial \text{Error}_{(total)}}{\partial W_{aa}} = \sum_{t=1}^{\text{end of Seq T}} \frac{\partial \text{Error}_{(t)}}{\partial W_{aa}} = \sum_{t=1}^{\text{end of Seq T}} \left[\sum_{k=1}^t \frac{\partial \text{Error}_t}{\partial Y_t} * \frac{\partial a_k}{\partial W_{aa}} * \frac{\partial \hat{Y}_t}{\partial a_t} * \prod_{j=k}^{t-1} \frac{\partial a_{j+1}}{\partial a_j} \right]$$

$a_{j+1} = \tanh(a_j * w_{aa} + x_{j+1} * w_{ax} + \text{bias}) \Rightarrow$ value of $\frac{\partial a_{j+1}}{\partial a_j}$ may be in range $[0 \dots 1] \Rightarrow$ Fraction

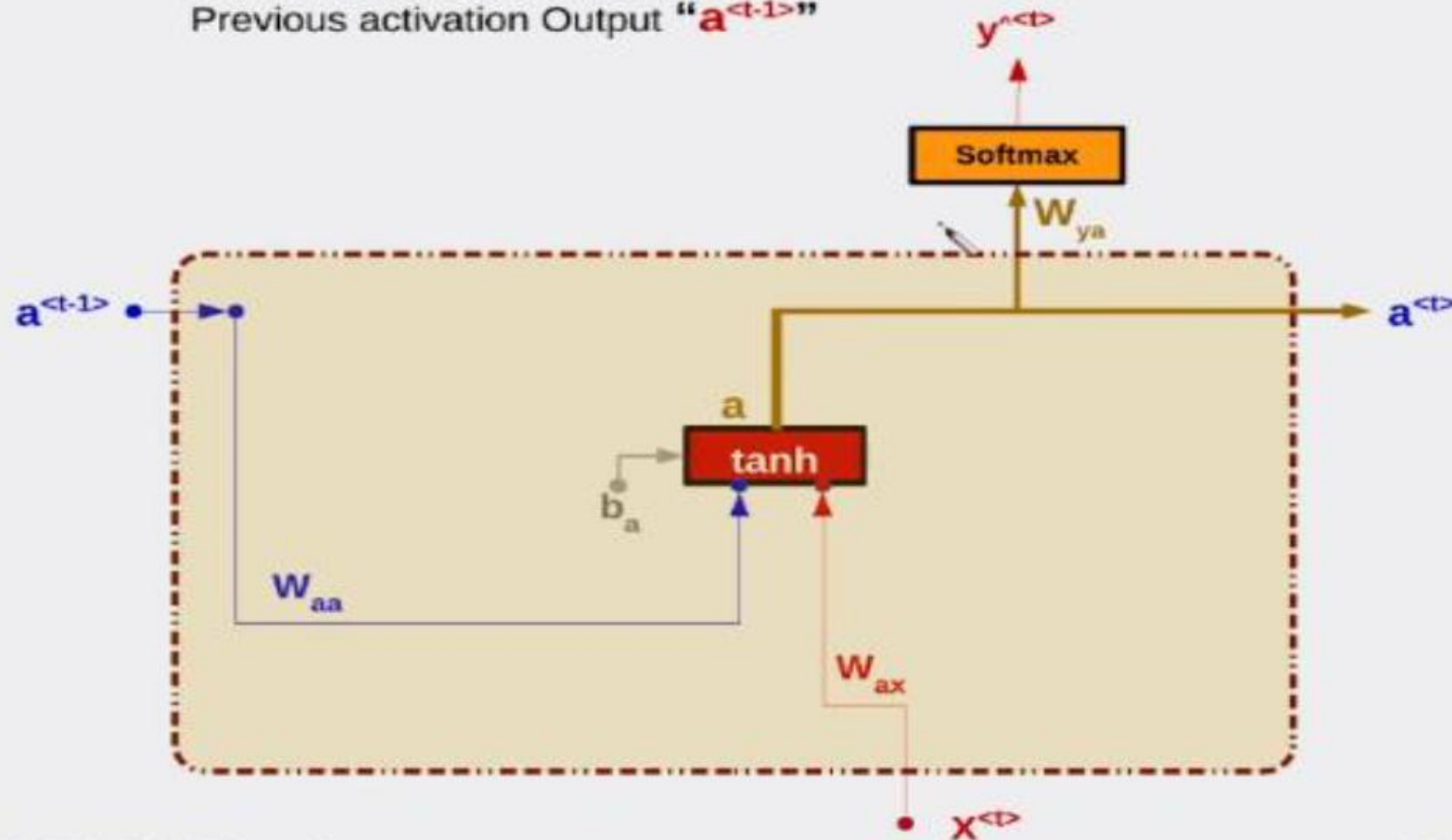
for $T=100$ timesteps, the longest length of fractions multiplication = $T - 1 = 99$ times $\Rightarrow$ Vanish

Remember: $\tanh(z) \Rightarrow [-1..1]$ $\frac{\partial \tanh(z)}{\partial (z)} = 1 - \tanh^2 z \Rightarrow$ **Min value=0** (at $\tanh(z) = 1$ or -1) **Max vlaue=1** (at $\tanh(z)=0$)

Recurrent Neural Network

Recurrent Neural Network (RNN)

Output of Activation Function at time t ; " $a^{<t>}$ " depends on BOTH:-
Input " $x^{<t>}$ " and
Previous activation Output " $a^{<t-1>}$ "

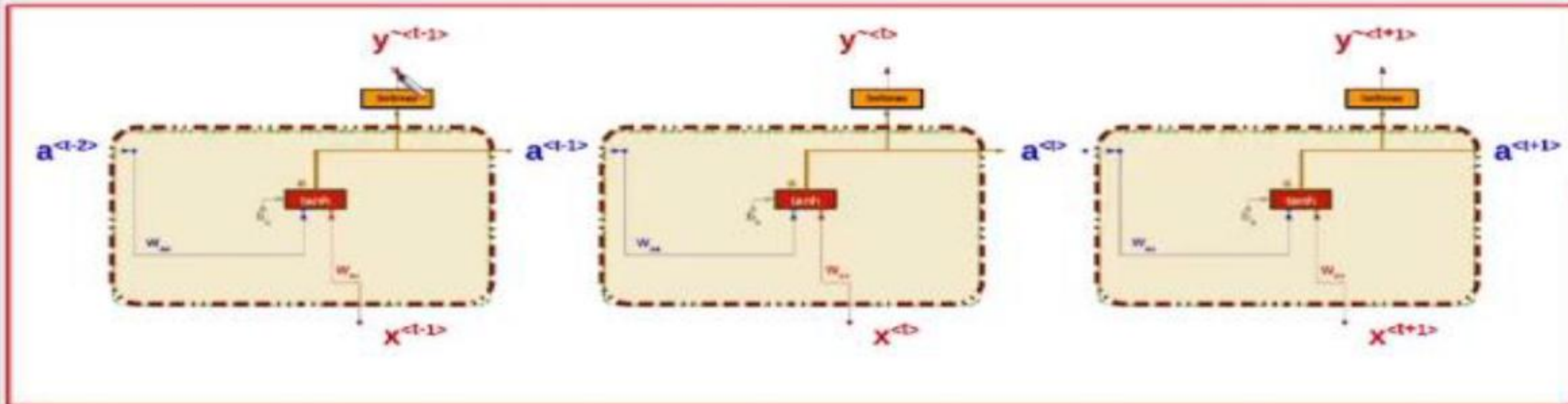


Recurrent Neural Network

Recurrent Neural Network (RNN)

Output of Activation Function at time t ; " $a^{<t>}$ " depends on BOTH:-
Input " $x^{<t>}$ " and
Previous activation Output " $a^{<t-1>}$ "

Inputs at time $t-1, t, t+1$



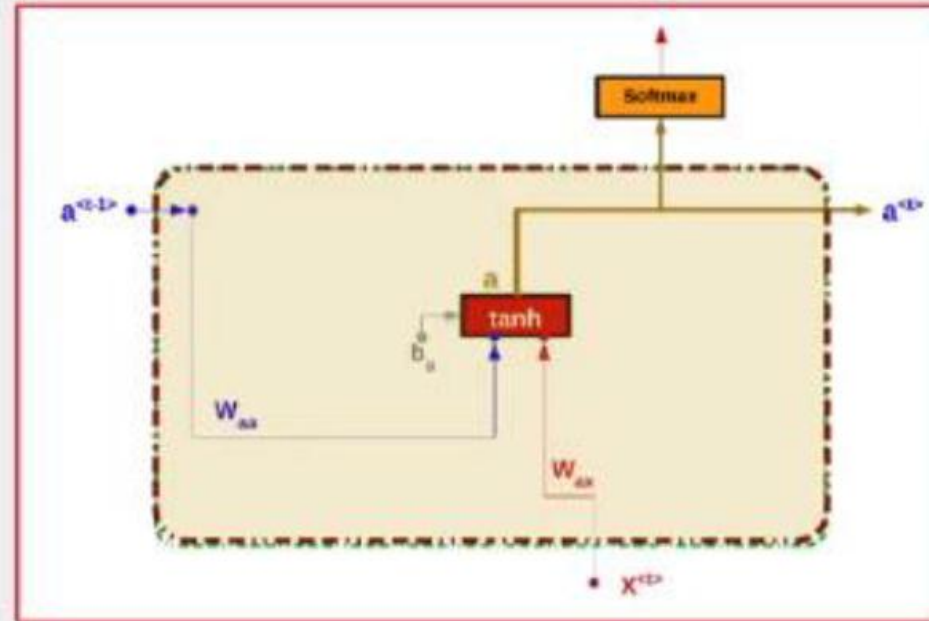
Recurrent Neural Network

Recurrent Neural Network (RNN)

$$a^{<1>} = F^n \left(\overbrace{W_{aa} a^{<0>}}^{\text{From Prev.}} + \overbrace{W_{ax} x^{<1>}}^{\text{Current I/P}} + b_a \right)$$

$$y^{<1>} = F^n (W_{ya} a^{<1>} + b_y)$$

Non Linearity



Recurrent Neural Network

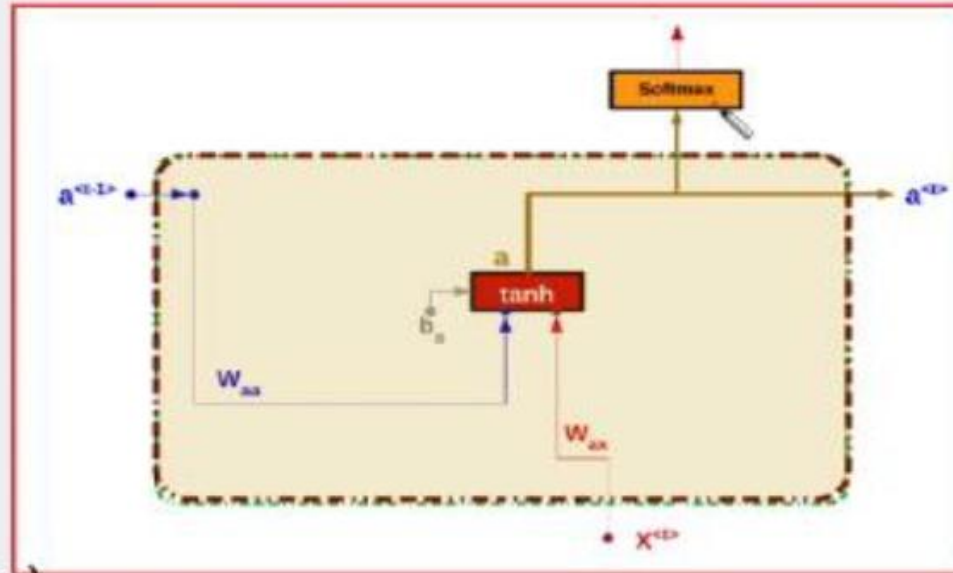
Recurrent Neural Network (RNN)

From Prev. Current I/P

$$a^{<1>} = F^n (W_{aa} a^{<0>} + W_{ax} x^{<1>} + b_a)$$

$$y^{<1>} = F^n (W_{ya} a^{<1>} + b_y)$$

Non Linearity



$$a^{<1>} = \tanh (W_{aa} a^{<0>} + W_{ax} x^{<1>} + b_a) \quad \text{May be tanh, ReLU, ...}$$

$$y^{<1>} = \text{Softmax} (W_{ya} a^{<1>} + b_y) \quad \text{Segmoid for Binary O/P, Softmax for Multi-Class O/P}$$

$$a^{<t>} = \tanh (W_{aa} a^{<t-1>} + W_{ax} x^{<t>} + b_a) \quad \text{May be tanh, ReLU, ...}$$

$$y^{<t>} = \text{Softmax} (W_{ya} a^{<t>} + b_y) \quad \text{Segmoid for Binary O/P, Softmax for Multi-Class O/P}$$

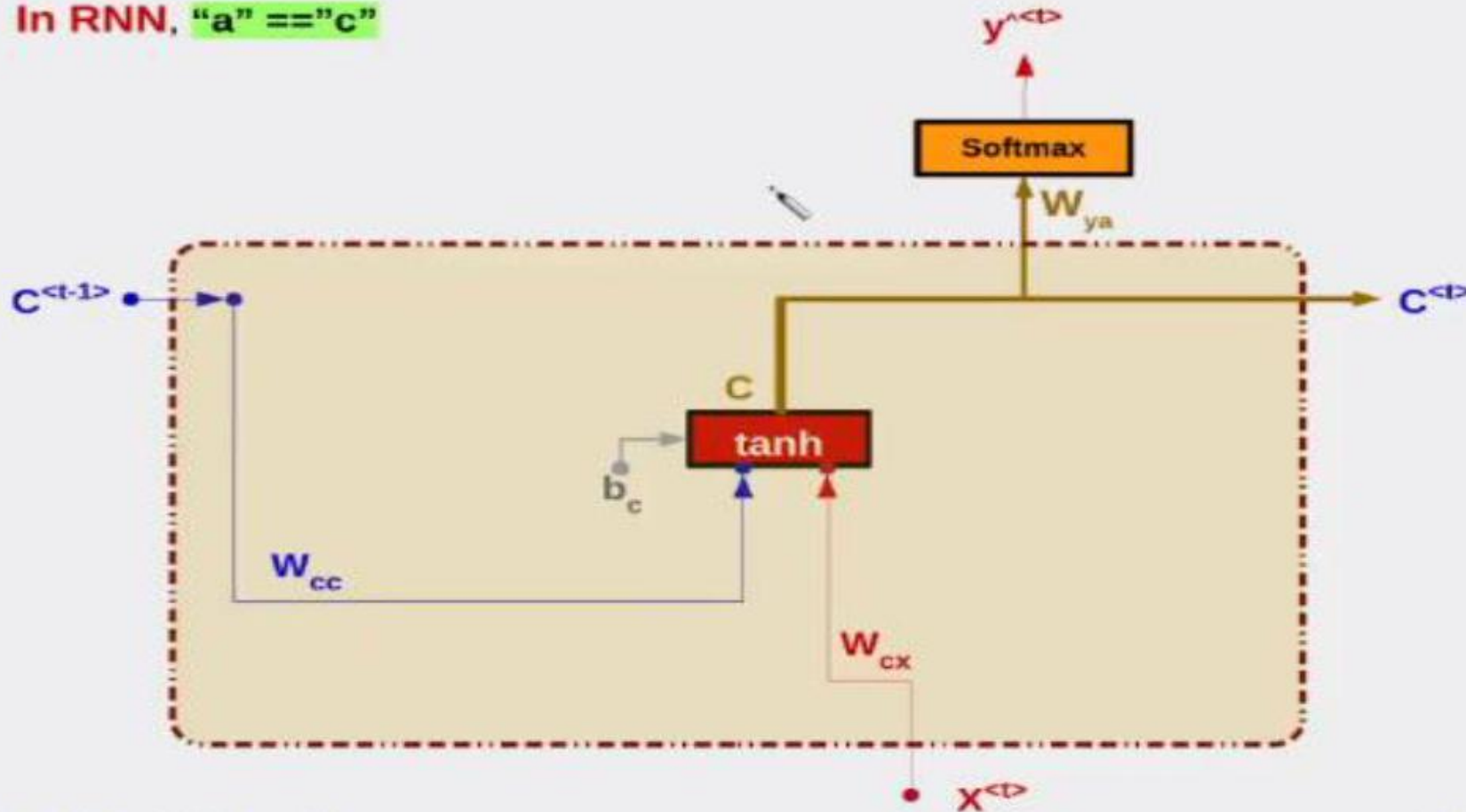
[2] Gated Recurrent Unit (GRU)

Recurrent Neural Network

From RNN to GRU

Define **Memory Cell "C"** in addition to Output of **Activation function "a"**.

In RNN, "a" == "c"



Recurrent Neural Network

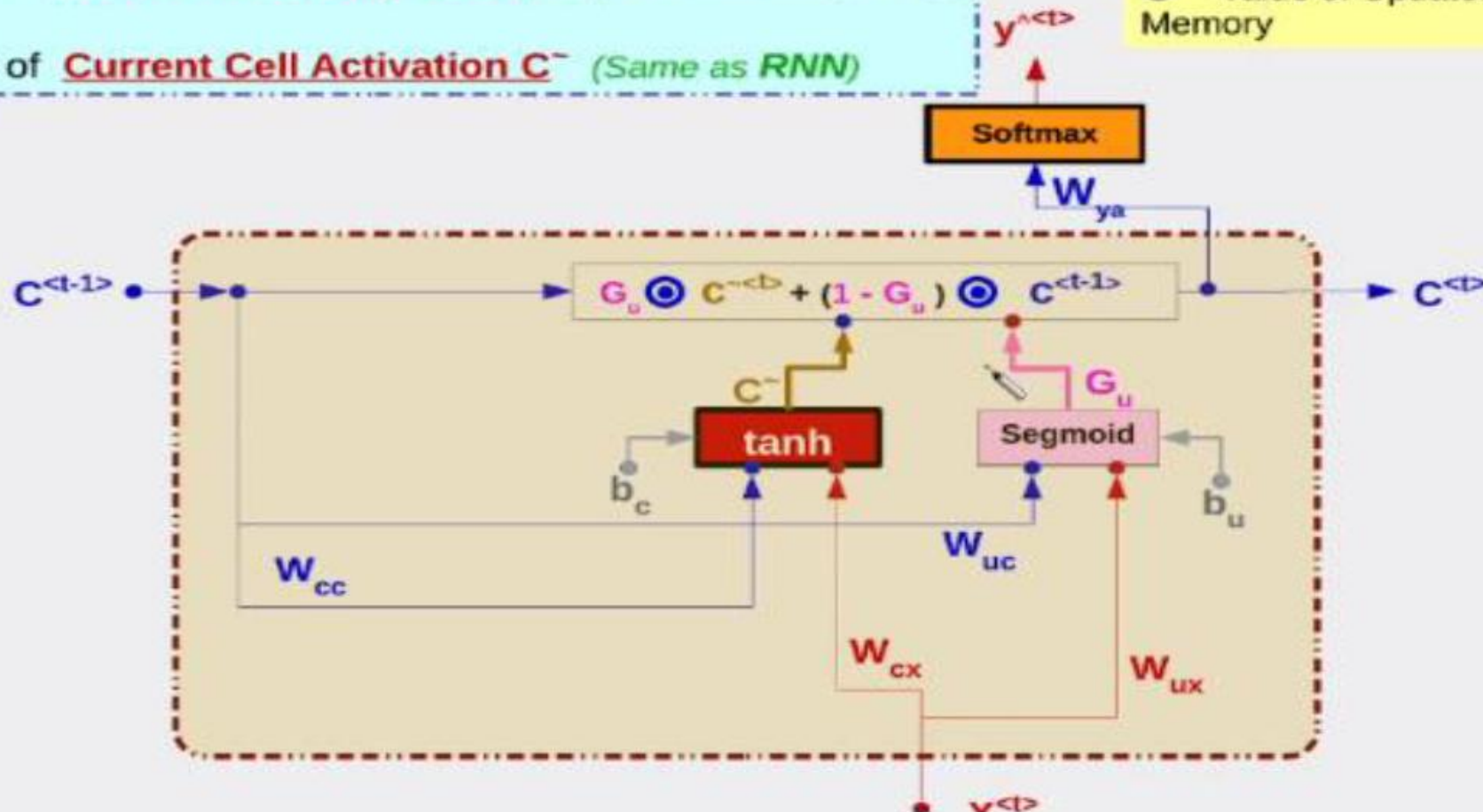
From RNN to GRU

[1] Adding Update Gate G_u

Value of CURRENT Memory Cell $C^{<t>}$ =
Percentage of Previous Memory Cell $C^{<t-1>}$ (Not Affected by $X^{<t>}$)
+
Percentage of Current Cell Activation C^- (Same as RNN)

C^- Candidate Value
of Updated Memory

C Value of Updated
Memory



Recurrent Neural Network

From RNN to GRU

[1] Adding Update Gate G_u

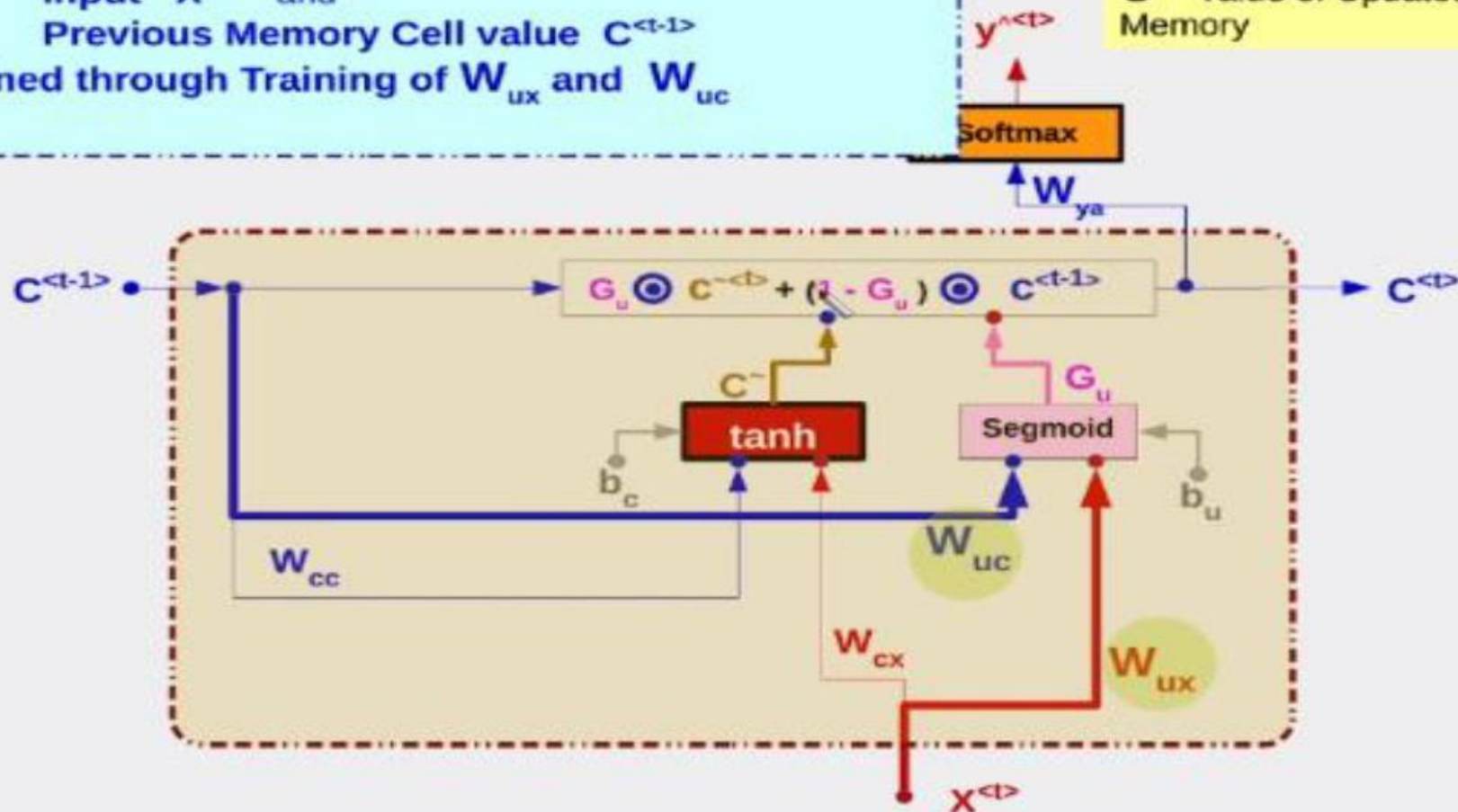
Value of G_u is based on:

Input $X^{<t>}$ and
Previous Memory Cell value $C^{<t-1>}$

G_u is obtained through Training of W_{ux} and W_{uc}

C^- Candidate Value
of Updated Memory

C Value of Updated
Memory



Recurrent Neural Network

From RNN to GRU

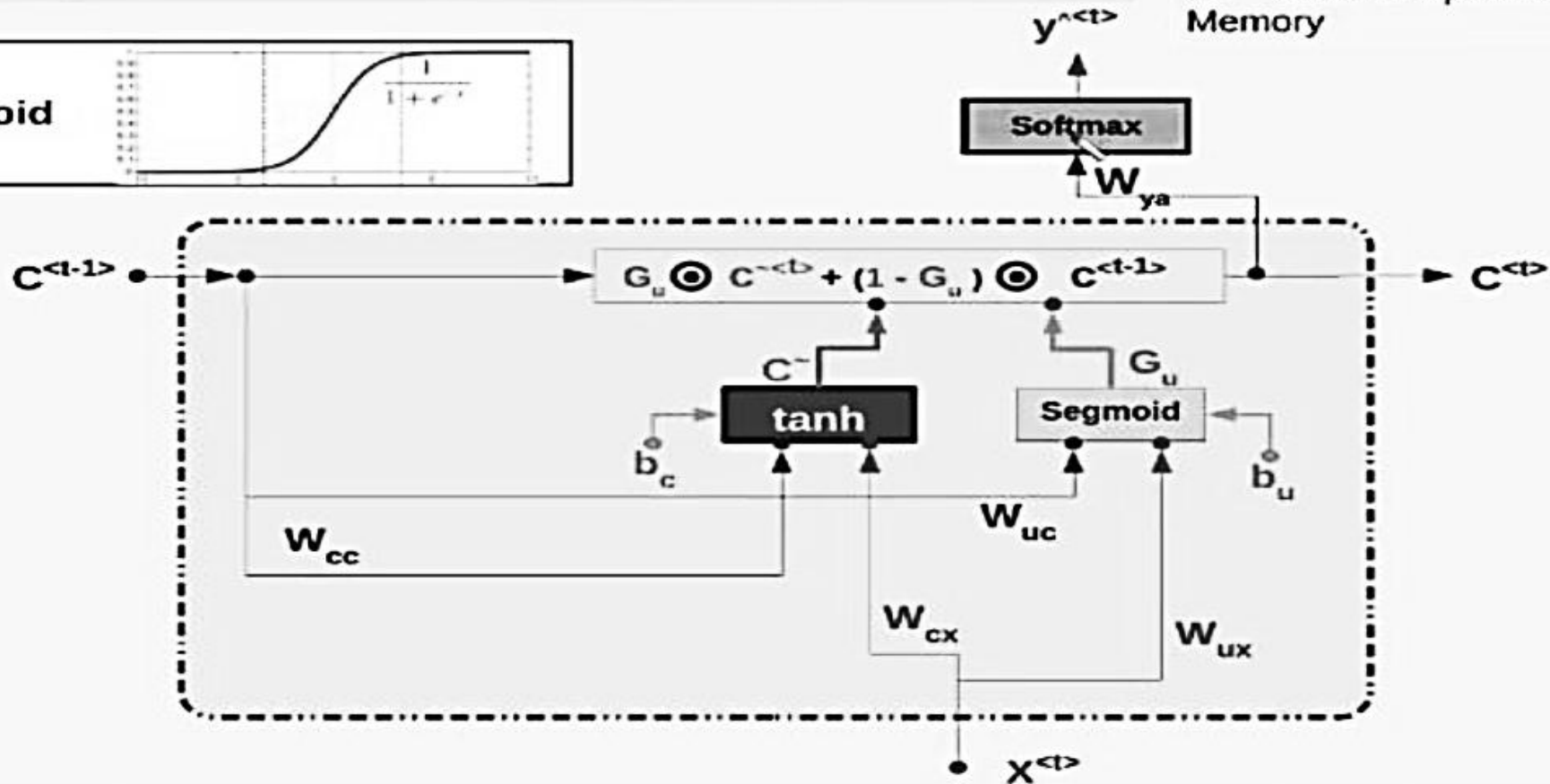
[1] Adding Update Gate G_u

If $G_u = 0$, Keep Memory Value " $C^{<t>}$ " Same as Previous Value " $C^{<t-1>}$ "

If $G_u = 1$, Forget Previous Memory Value " $C^{<t-1>}$ "

C^- Candidate Value of Updated Memory

C Value of Updated Memory



Recurrent Neural Network

From RNN to GRU

$$G_u = \text{Sigmoid}(W_{uc}c^{<t-1>} + W_{ux}x^{<t>} + b_u)$$

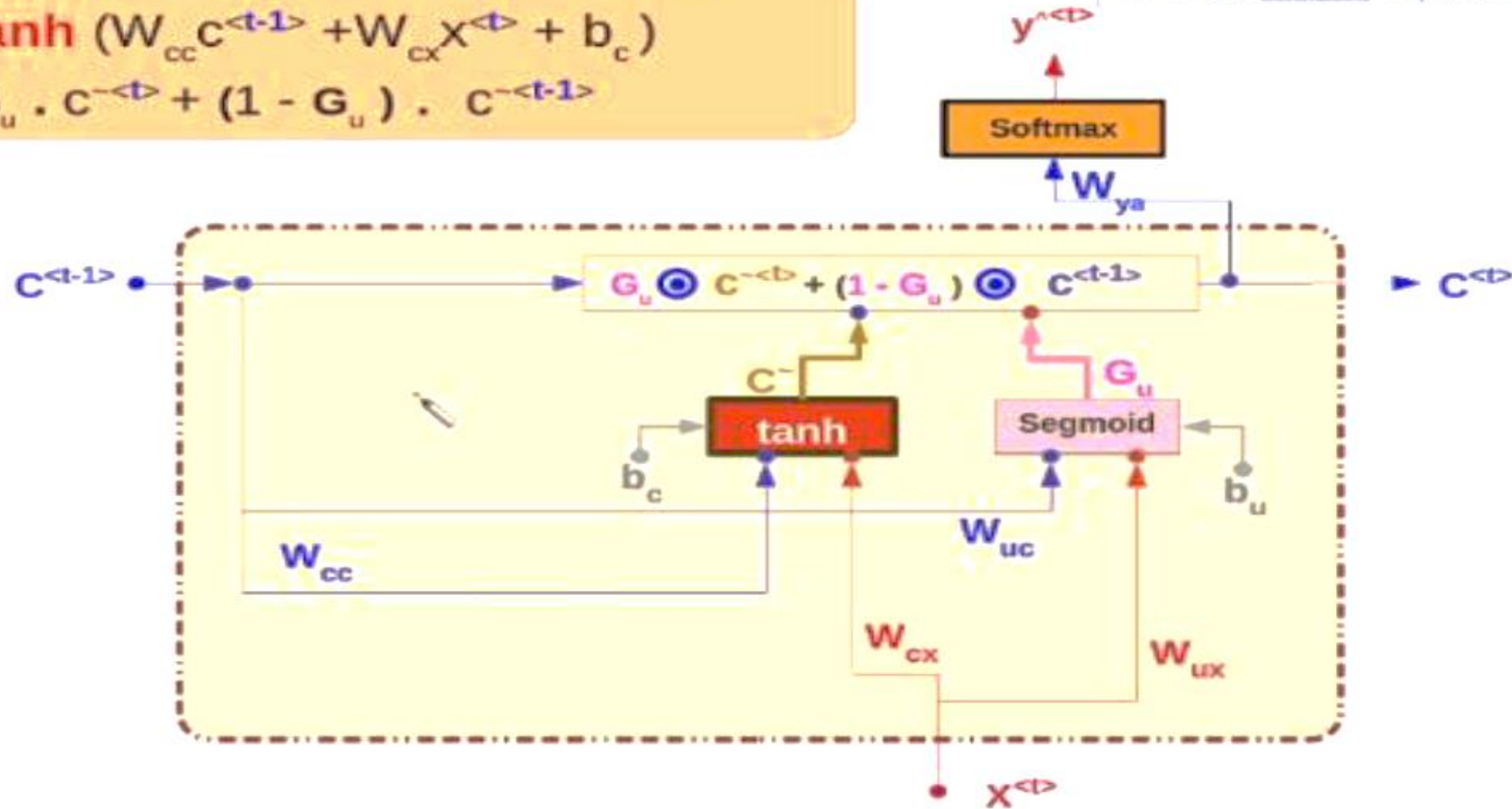
$$c^{-<t>} = \tanh(W_{cc}c^{<t-1>} + W_{cx}x^{<t>} + b_c)$$

$$c^{<t>} = G_u \cdot c^{-<t>} + (1 - G_u) \cdot c^{<t-1>}$$

c^{-} is the Candidate Update

G_u is the Update Gate

C is the Actual Update

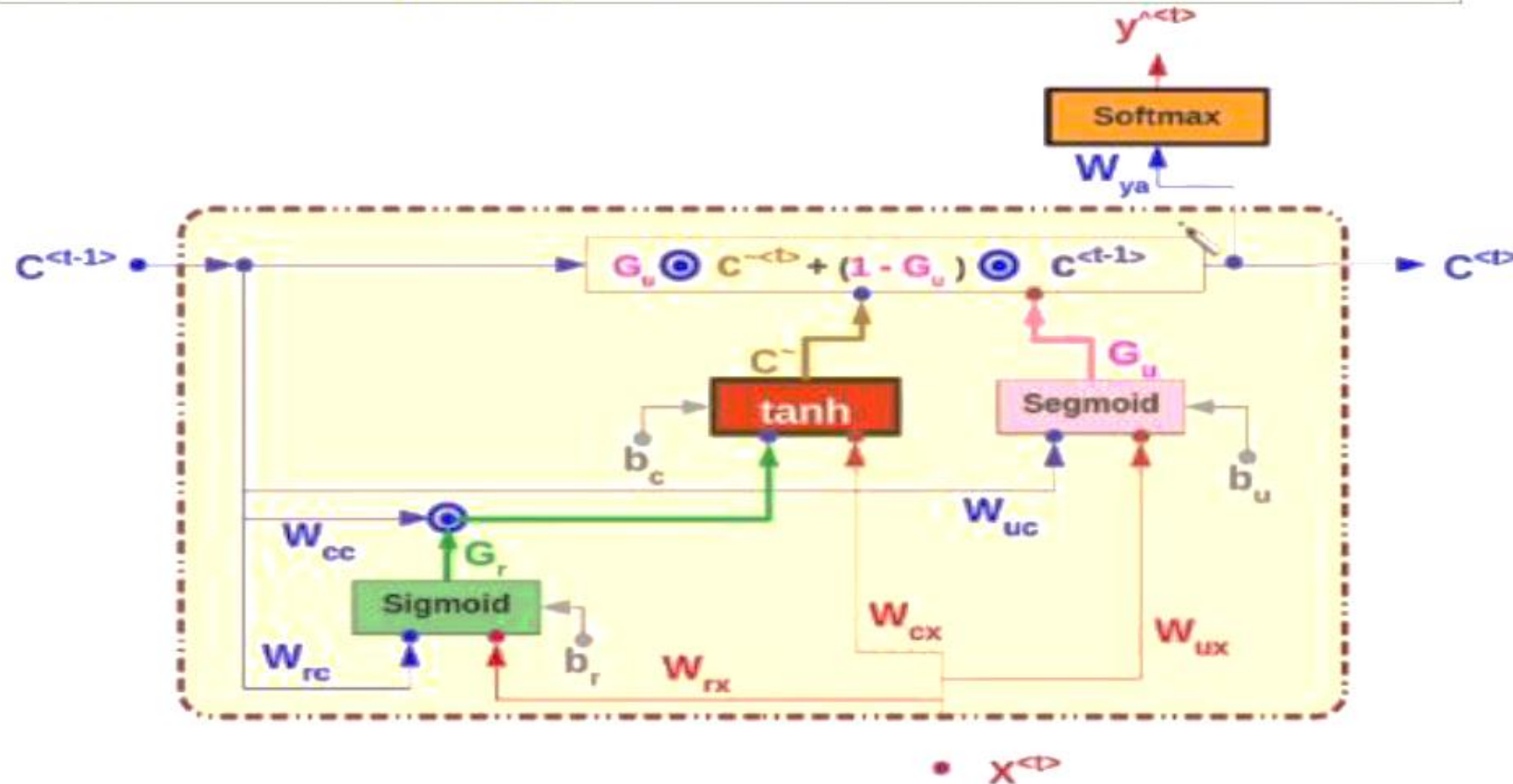


Recurrent Neural Network

From RNN to GRU

[2] Adding Relevance Gate G_r

If $G_r = 1$, $C^{<t-1>}$ is **Relevant** to update Candidate Memory cell value " C^- "
If $G_r = 0$, $C^{<t-1>}$ is **Irrelevant** to update Candidate Memory cell value " C^- "



Recurrent Neural Network

From RNN to GRU

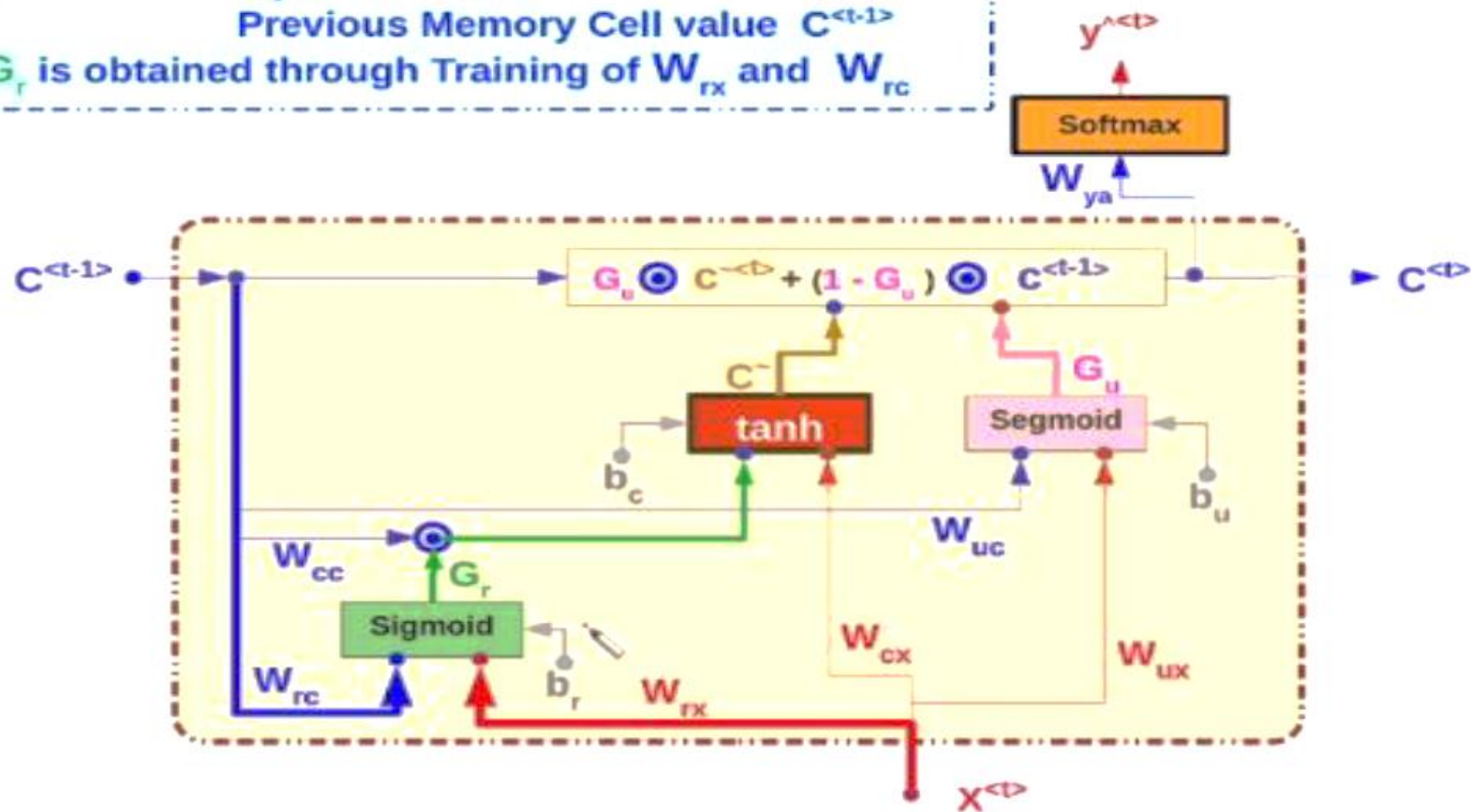
[2] Adding Relevance Gate G_r

Value of G_r is based on:

Input $X^{<t>}$ and

Previous Memory Cell value $C^{<t-1>}$

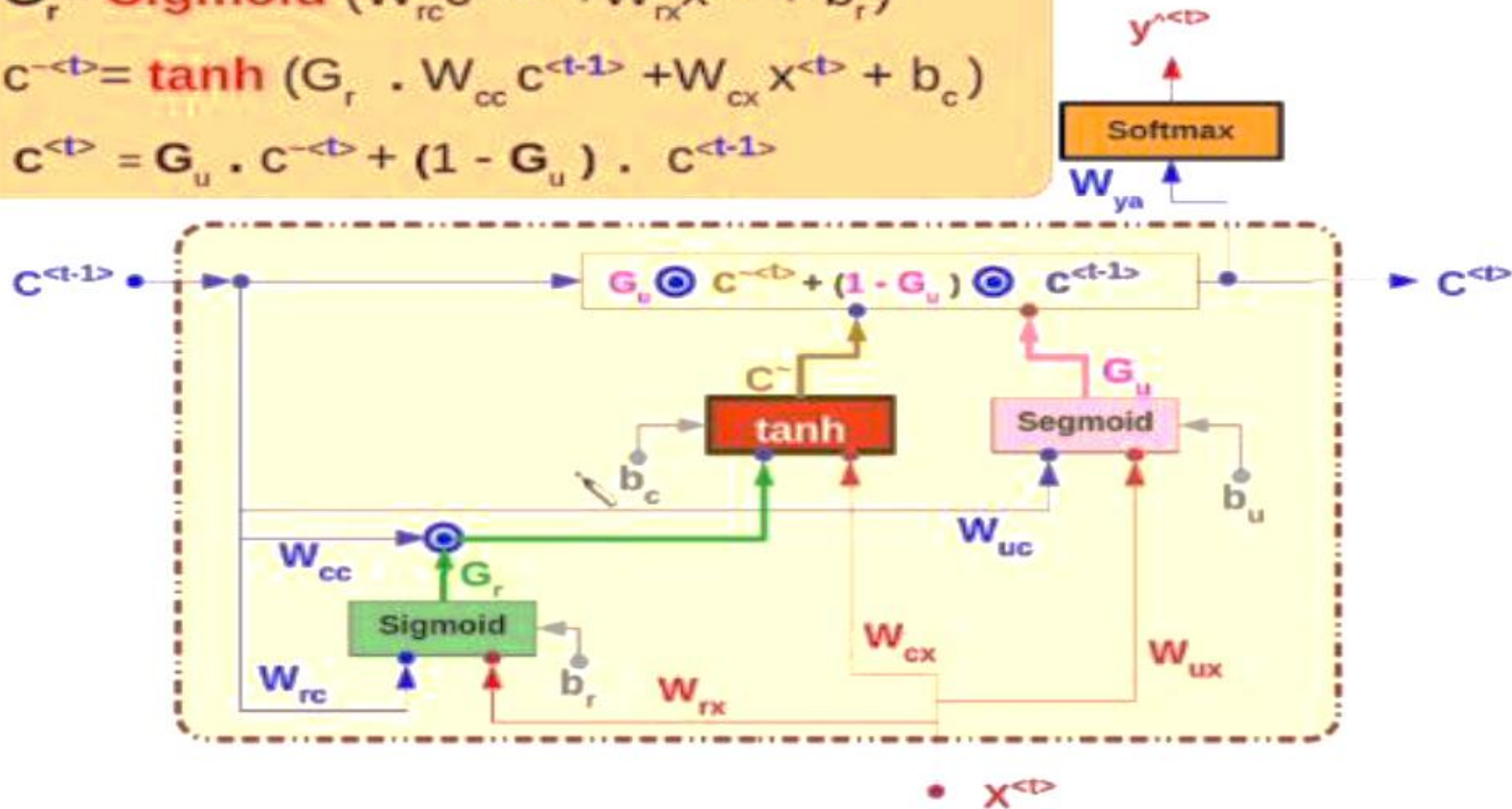
G_r is obtained through Training of W_{rx} and W_{rc}



Recurrent Neural Network

From RNN to GRU

$$\begin{aligned}G_u &= \text{Sigmoid}(W_{uc}c^{<t-1>} + W_{ux}x^{<t>} + b_u) \\G_r &= \text{Sigmoid}(W_{rc}c^{<t-1>} + W_{rx}x^{<t>} + b_r) \\c^{-<t>} &= \tanh(G_r \cdot W_{cc}c^{<t-1>} + W_{cx}x^{<t>} + b_c) \\c^{<t>} &= G_u \cdot c^{-<t>} + (1 - G_u) \cdot c^{<t-1>}\end{aligned}$$



Recurrent Neural Network

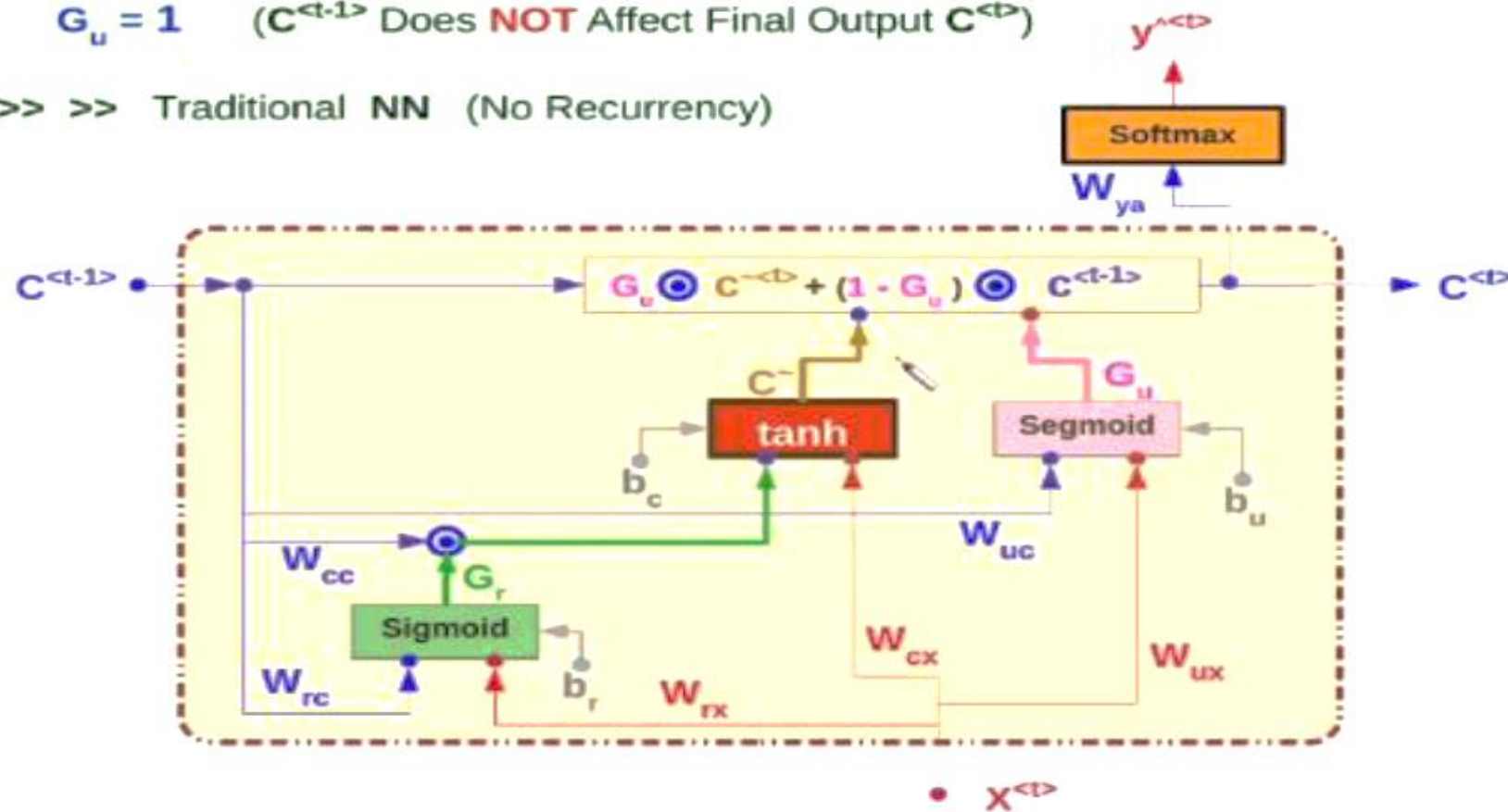
From RNN to GRU

IF

$G_r = 0$ (C^{t-1} Does **NOT** Affect tanh Output C^-)

$G_u = 1$ (C^{t-1} Does **NOT** Affect Final Output C^t)

>> >> Traditional NN (No Recurrency)



Recurrent Neural Network

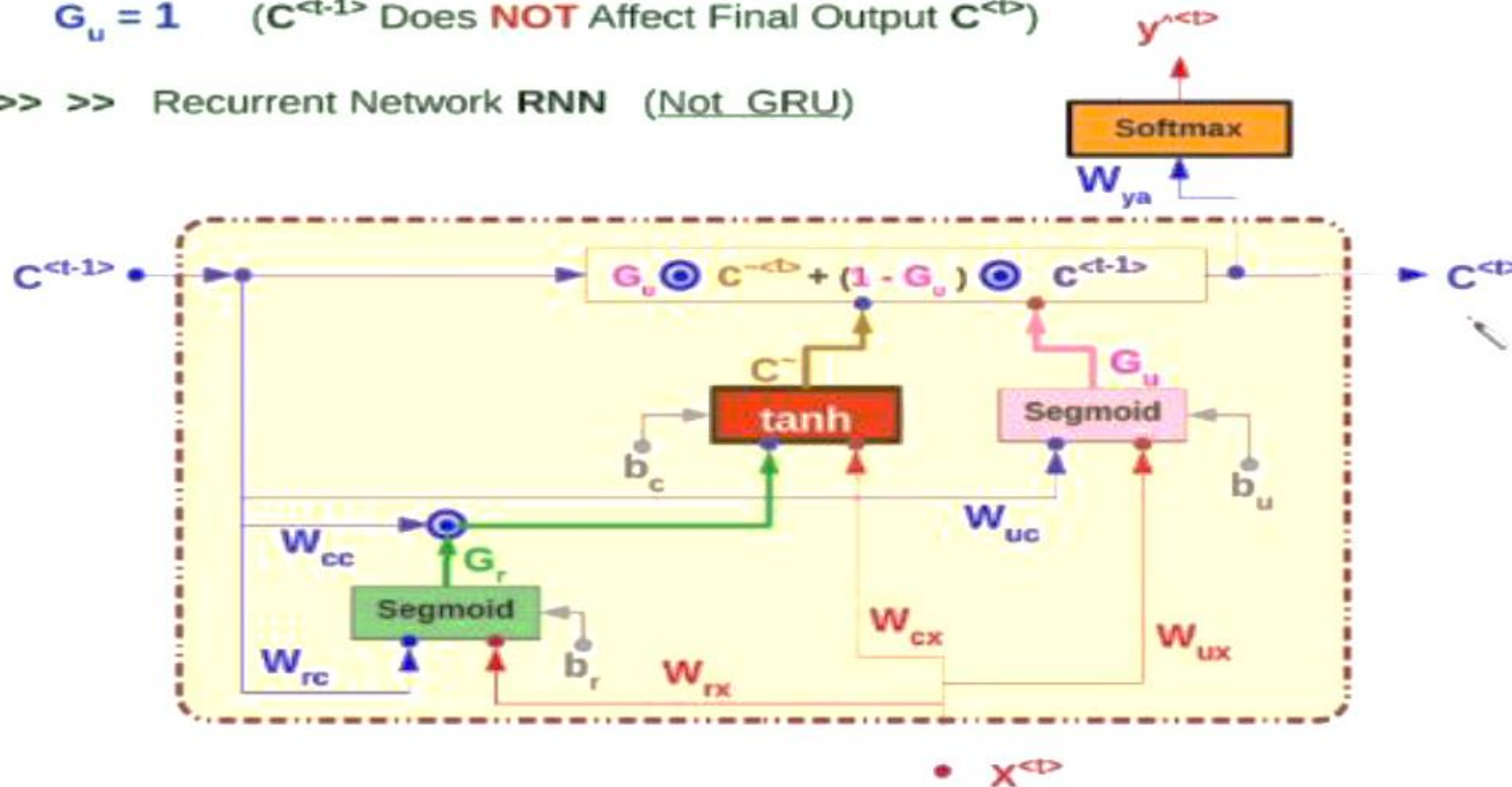
From RNN to GRU

IF

$G_r = 1$ ($C^{<t-1>}$ Affects tanh Output C^-)

$G_u = 1$ ($C^{<t-1>}$ Does **NOT** Affect Final Output $C^{<t>}$)

>> >> Recurrent Network **RNN** (Not GRU)



Recurrent Neural Network

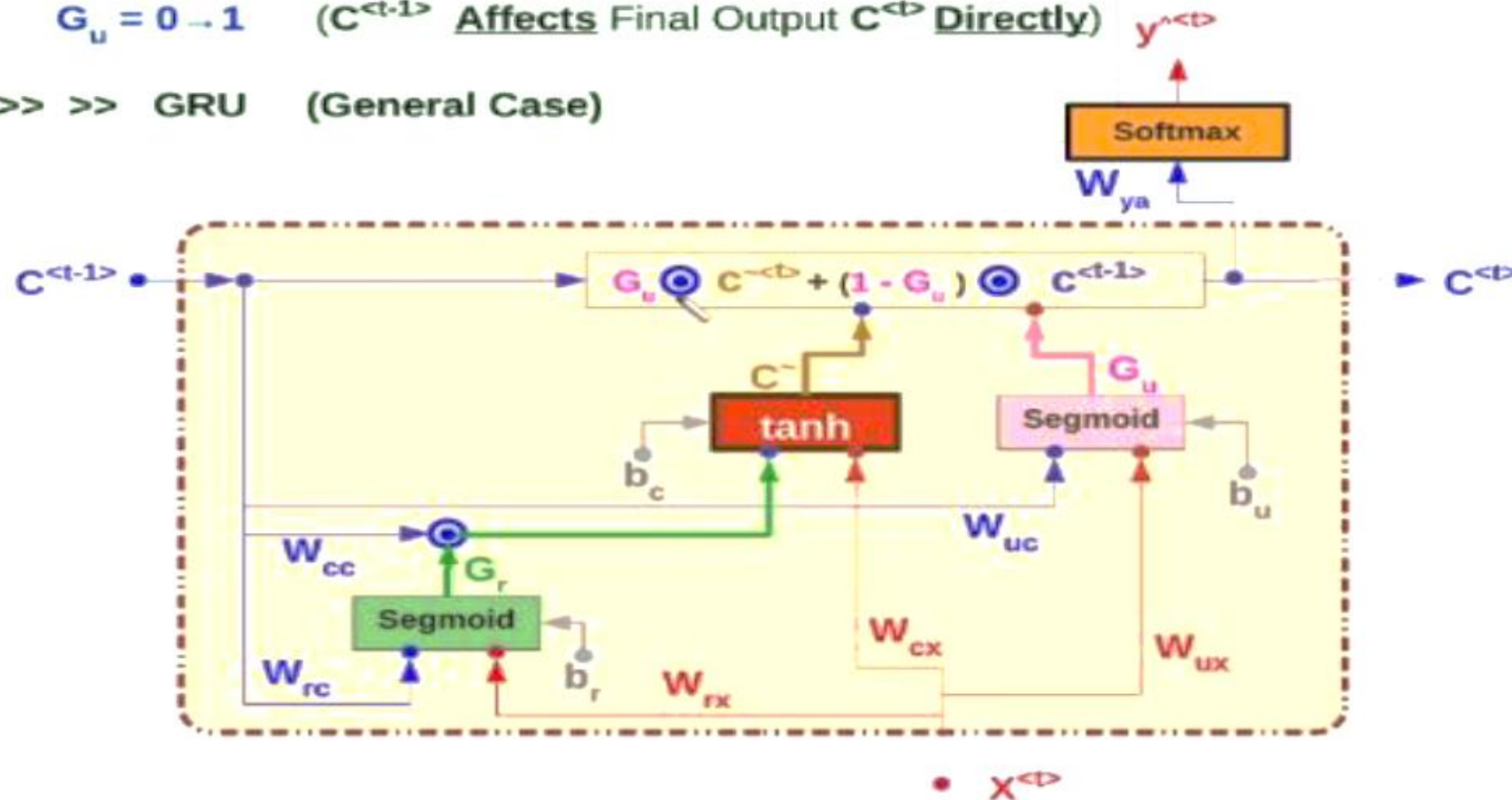
From RNN to GRU

IF

$G_r = 0 \rightarrow 1$ ($C^{<t-1>}$ Affects tanh Output C^-)

$G_u = 0 \rightarrow 1$ ($C^{<t-1>}$ Affects Final Output $C^{<t>}$ Directly)

>> >> GRU (General Case)



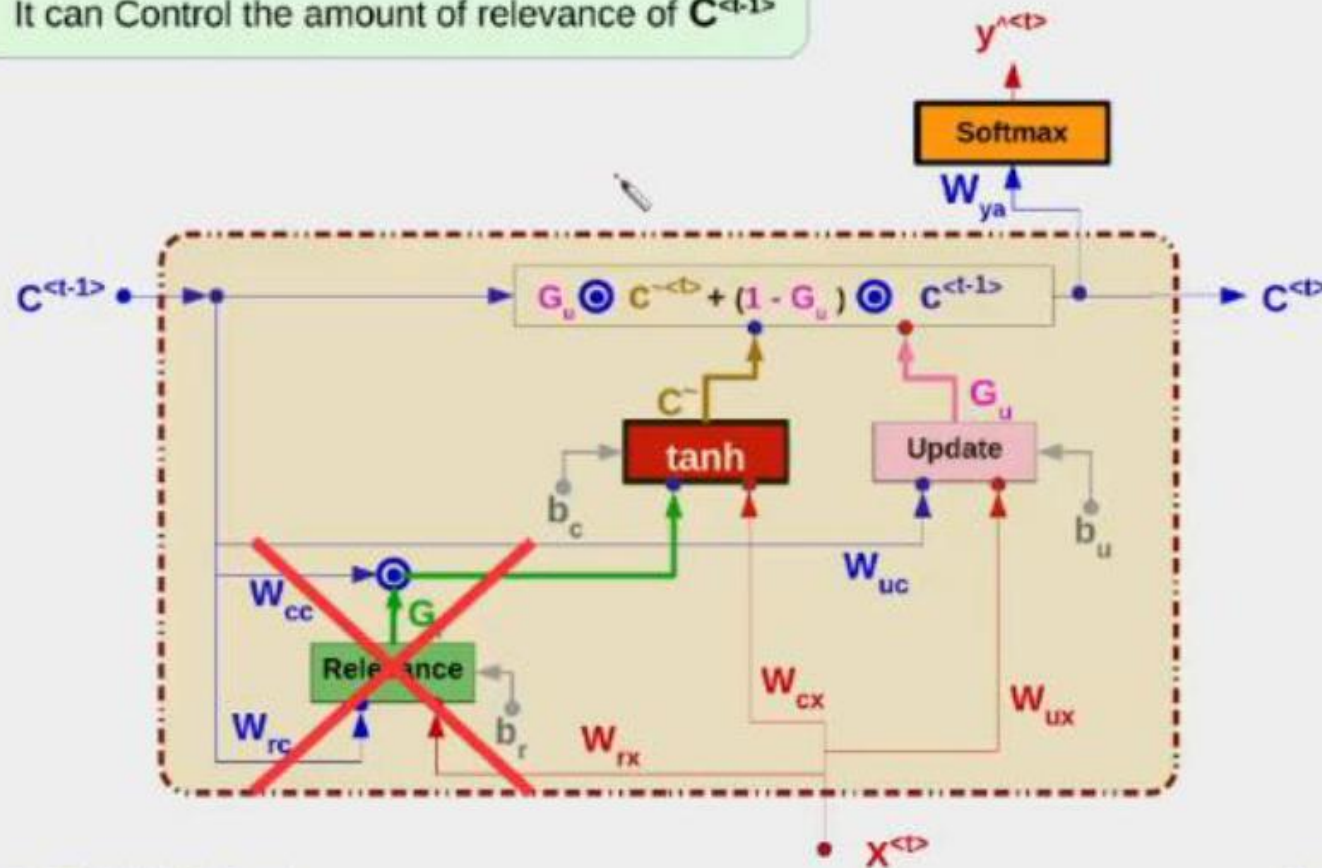
[3] Long Short Term Memory (LSTM)

Recurrent Neural Network

From GRU to LSTM

[1] Removing Relevance Gate G_r

W_{uc} can Do the same functionality
It can Control the amount of relevance of $C^{<t-1>}$



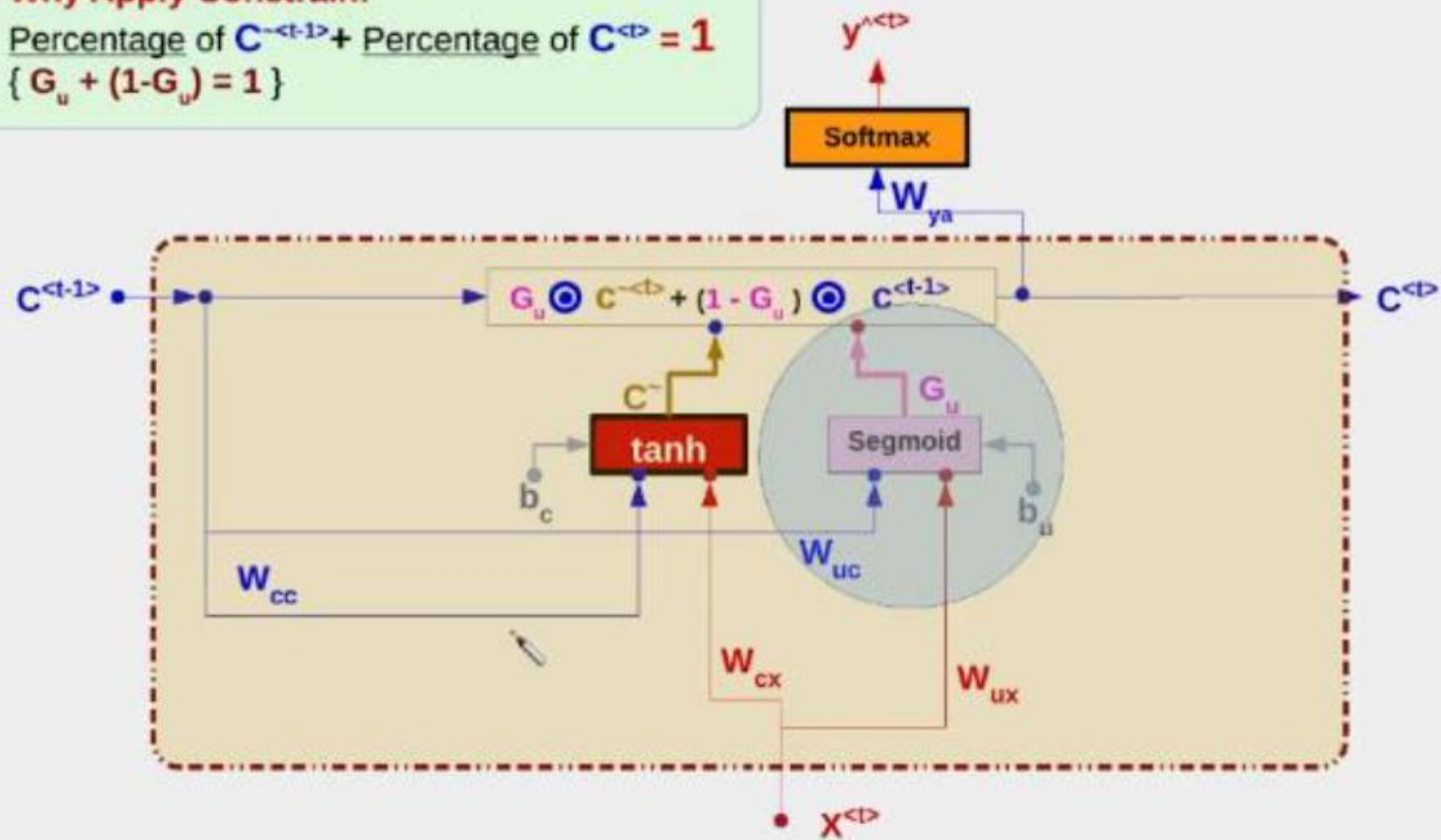
Recurrent Neural Network

From GRU to LSTM

[2] ??????

Why Apply Constrains:

Percentage of $C^{<t-1>}$ + Percentage of $C^{<t>} = 1$
 $\{ G_u + (1 - G_u) = 1 \}$



Recurrent Neural Network

From GRU to LSTM

[2] ??????

Why Apply Constrains:

Percentage of $C^{<t-1>}$ + Percentage of $C^{<t>}$ = 1
 $\{ G_u + (1 - G_u) = 1 \}$

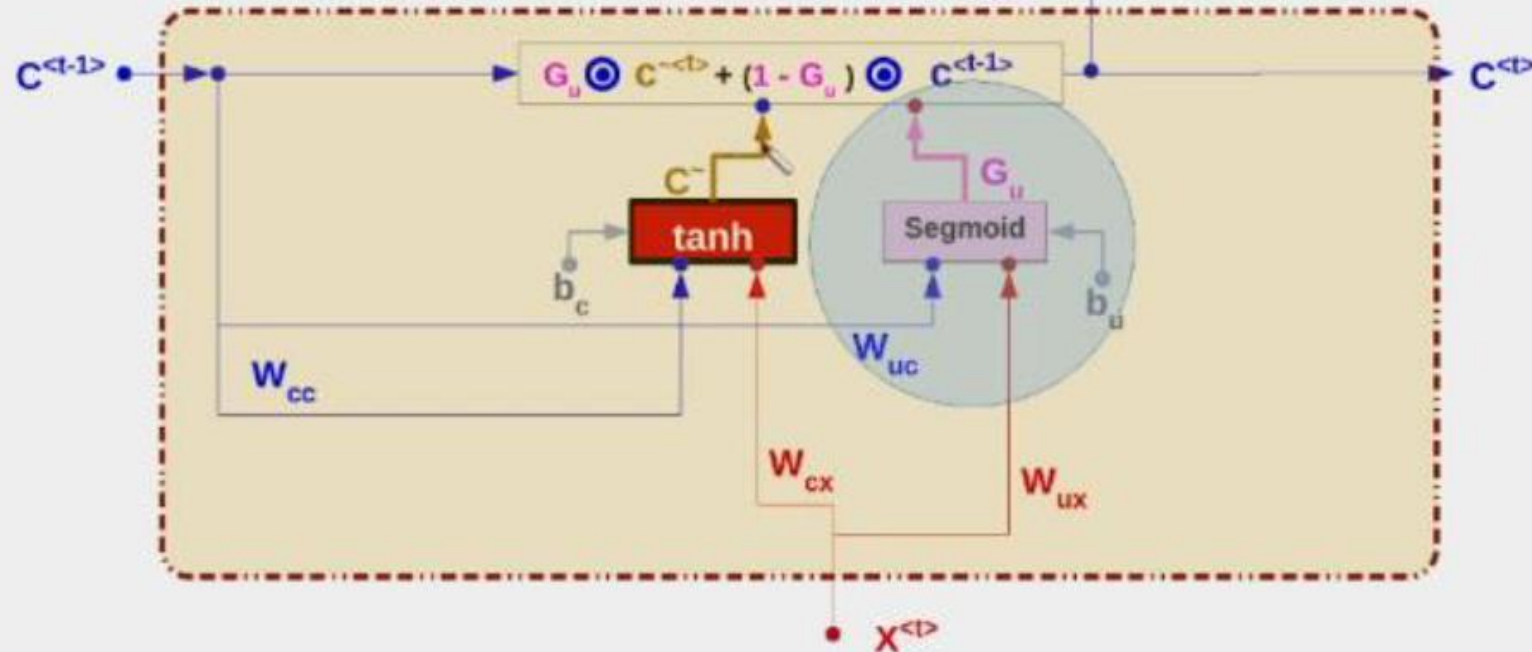
Answer

C^- value range $[-1 \rightarrow 1]$

C value range $[-1 \rightarrow 1]$

$\{ \tanh \text{ O/P range } [-1 \rightarrow 1] \}$

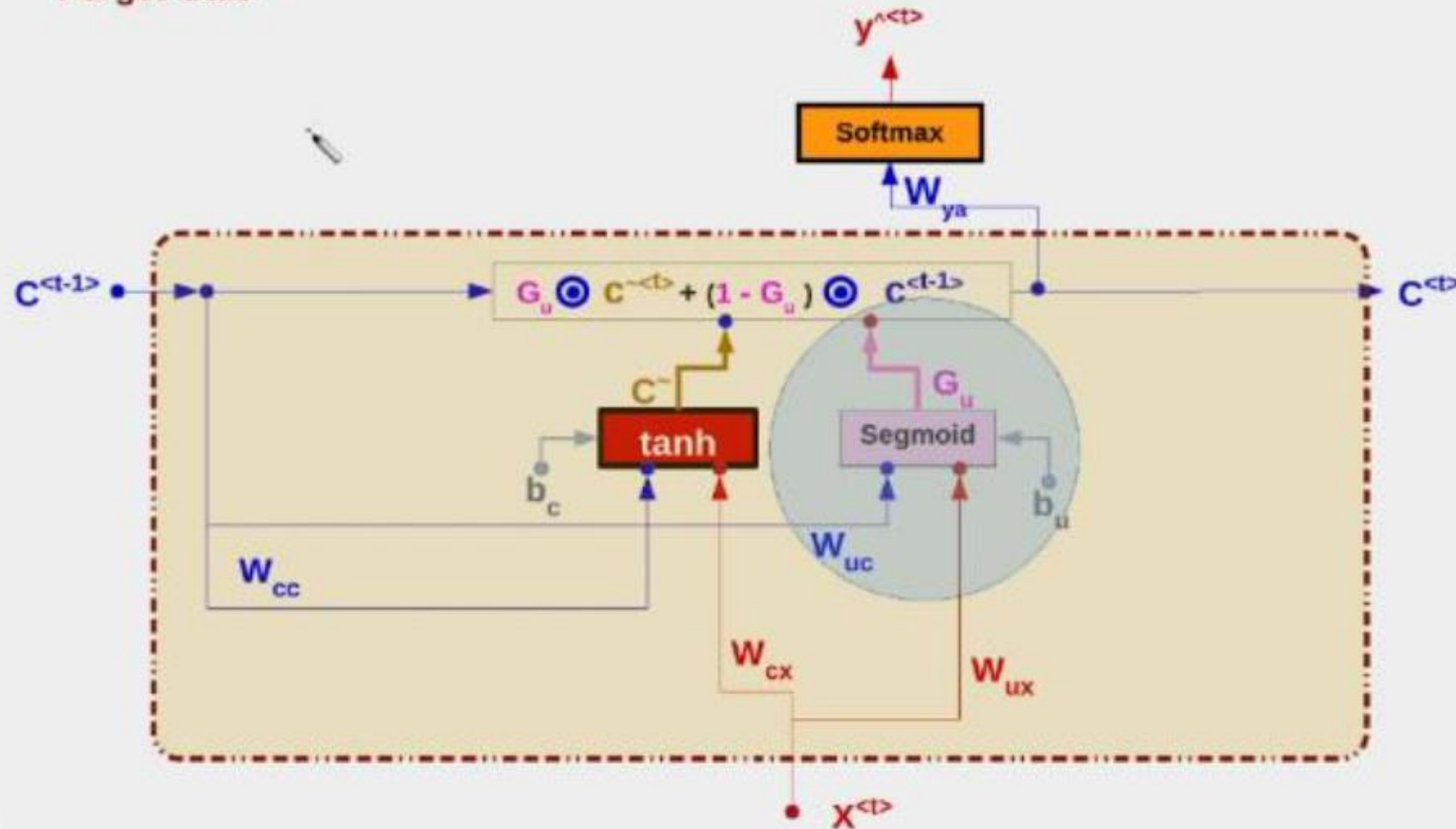
$\{ G_u + (1 - G_u) = 1 \}$



Recurrent Neural Network

From GRU to LSTM

[2] Split "Update Gate" into two gates:
"Update Gate"
"Forget Gate"



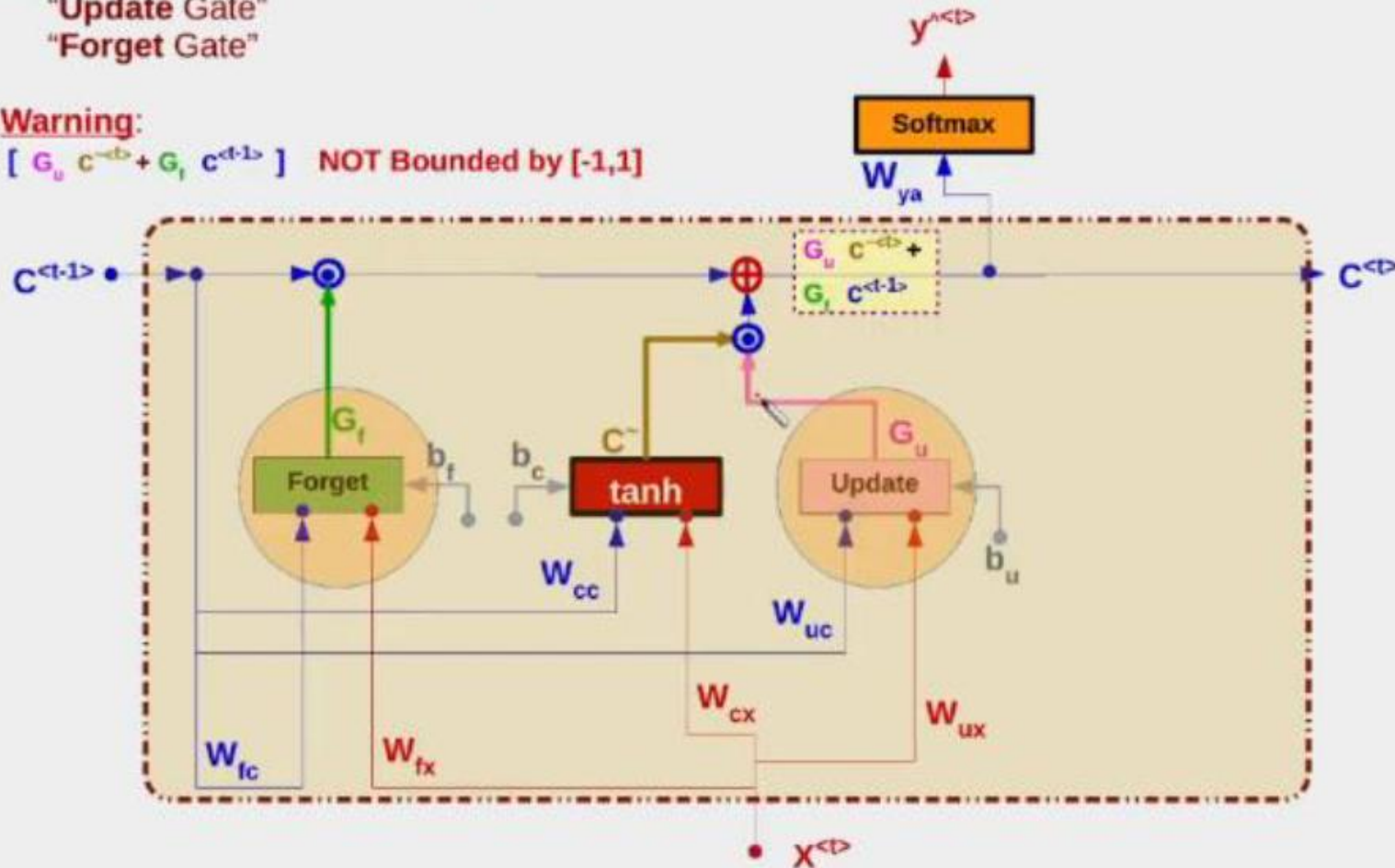
Recurrent Neural Network

From GRU to LSTM

[2] Split "Update Gate" into two gates:
"Update Gate"
"Forget Gate"

Warning:

$[G_u c^{<t-1>} + G_f c^{<t-1>}]$ NOT Bounded by $[-1,1]$



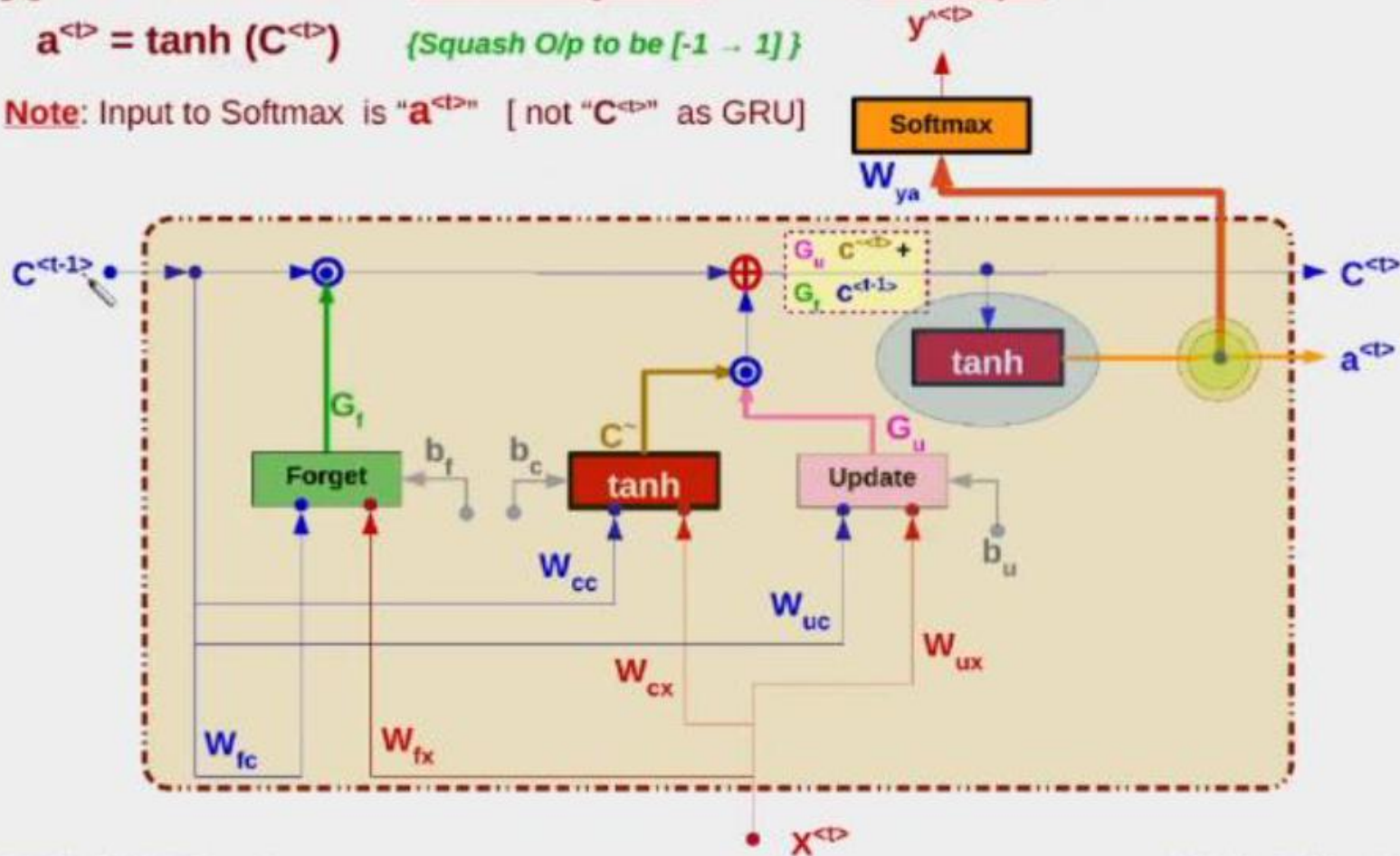
Recurrent Neural Network

From GRU to LSTM

[3] Differentiate between Cell Memory value $C^{<t>}$ and Cell Output $a^{<t>}$

$$a^{<t>} = \tanh(C^{<t>}) \quad \{ \text{Squash O/p to be } [-1 \rightarrow 1] \}$$

Note: Input to Softmax is " $a^{<t>}$ " [not " $C^{<t>}$ " as GRU]

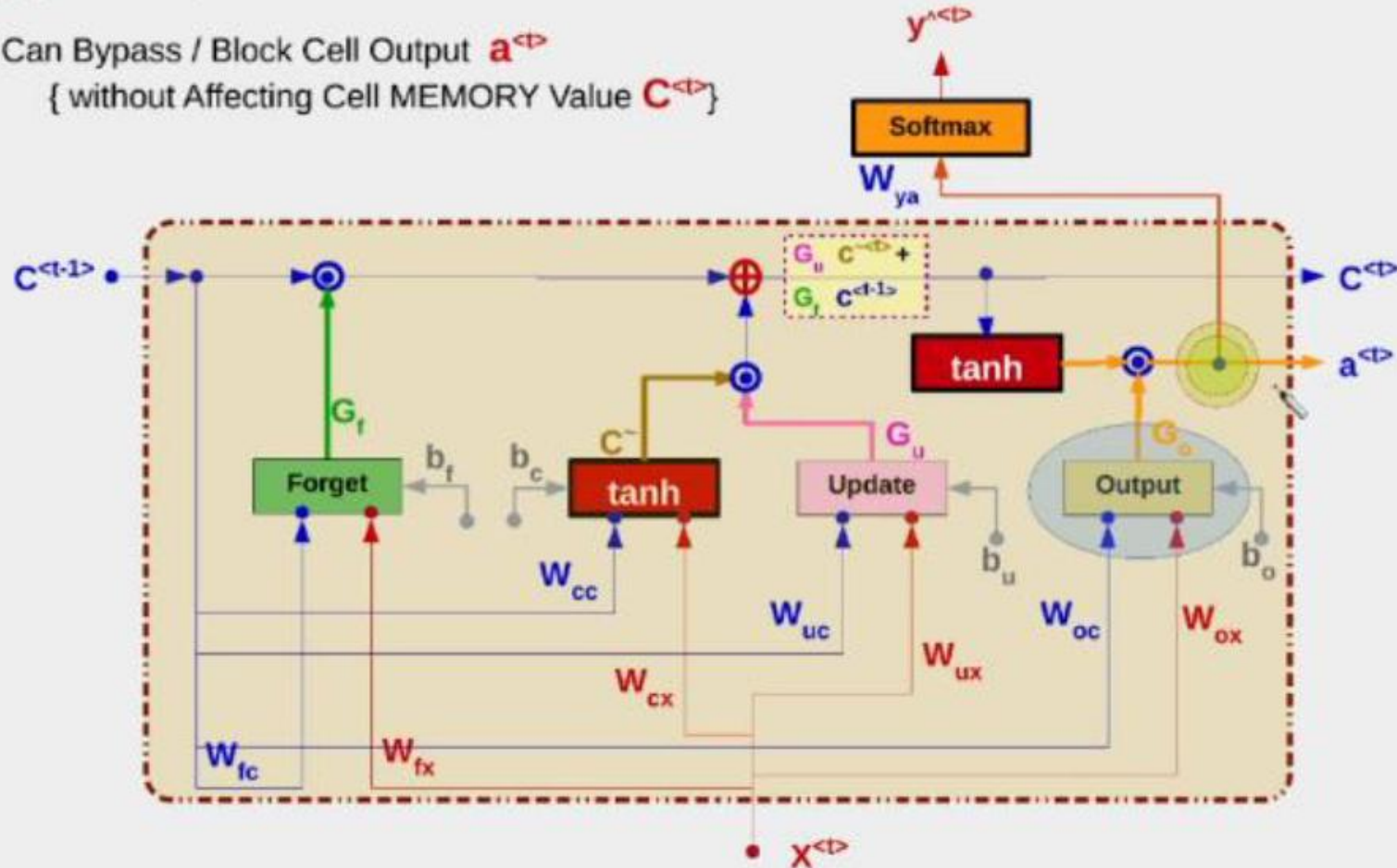


Recurrent Neural Network

From GRU to LSTM

[4] Add "Output Control Gate"

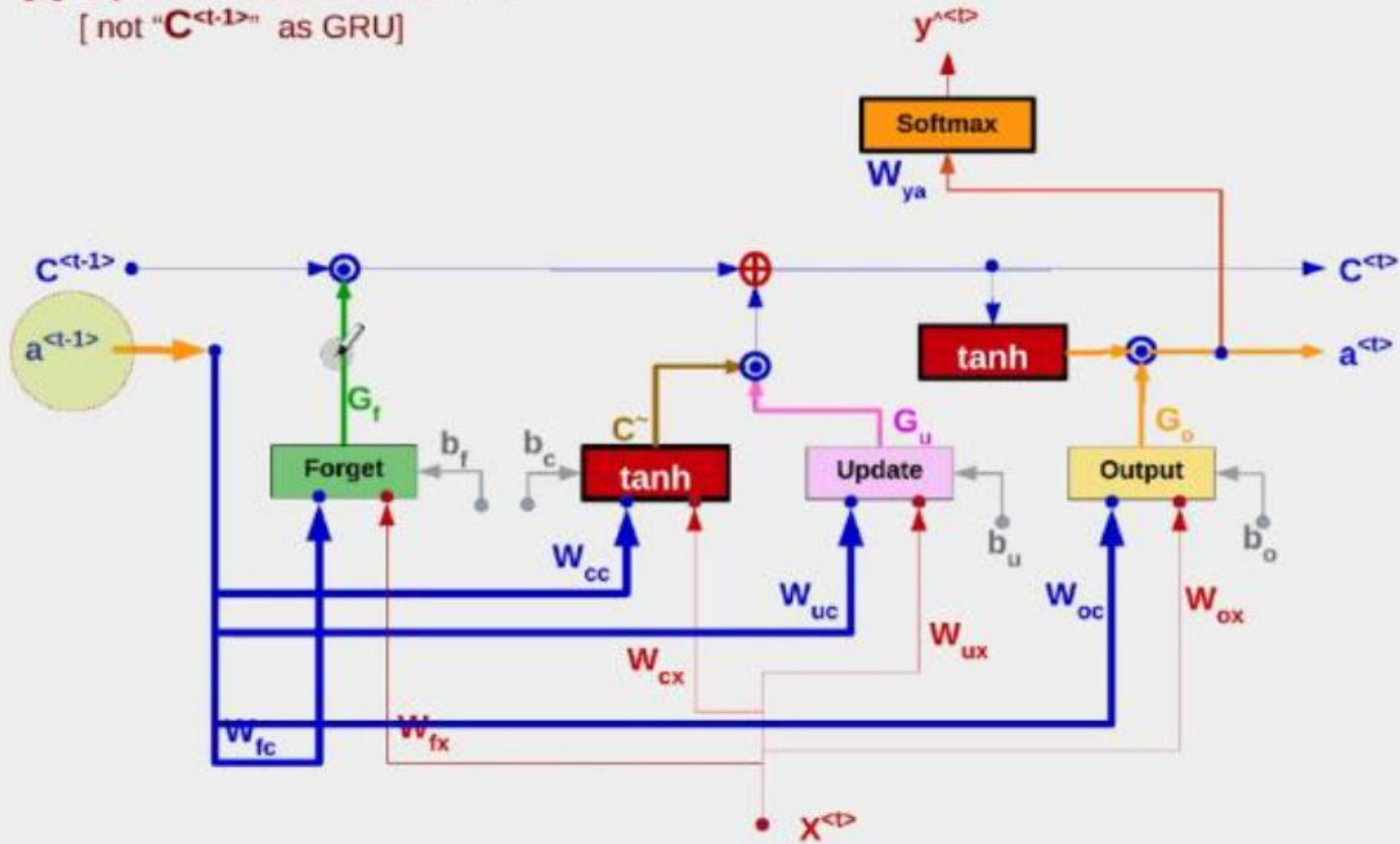
Can Bypass / Block Cell Output $a^{<t>}$
{ without Affecting Cell MEMORY Value $C^{<t>}$ }



Recurrent Neural Network

From GRU to LSTM

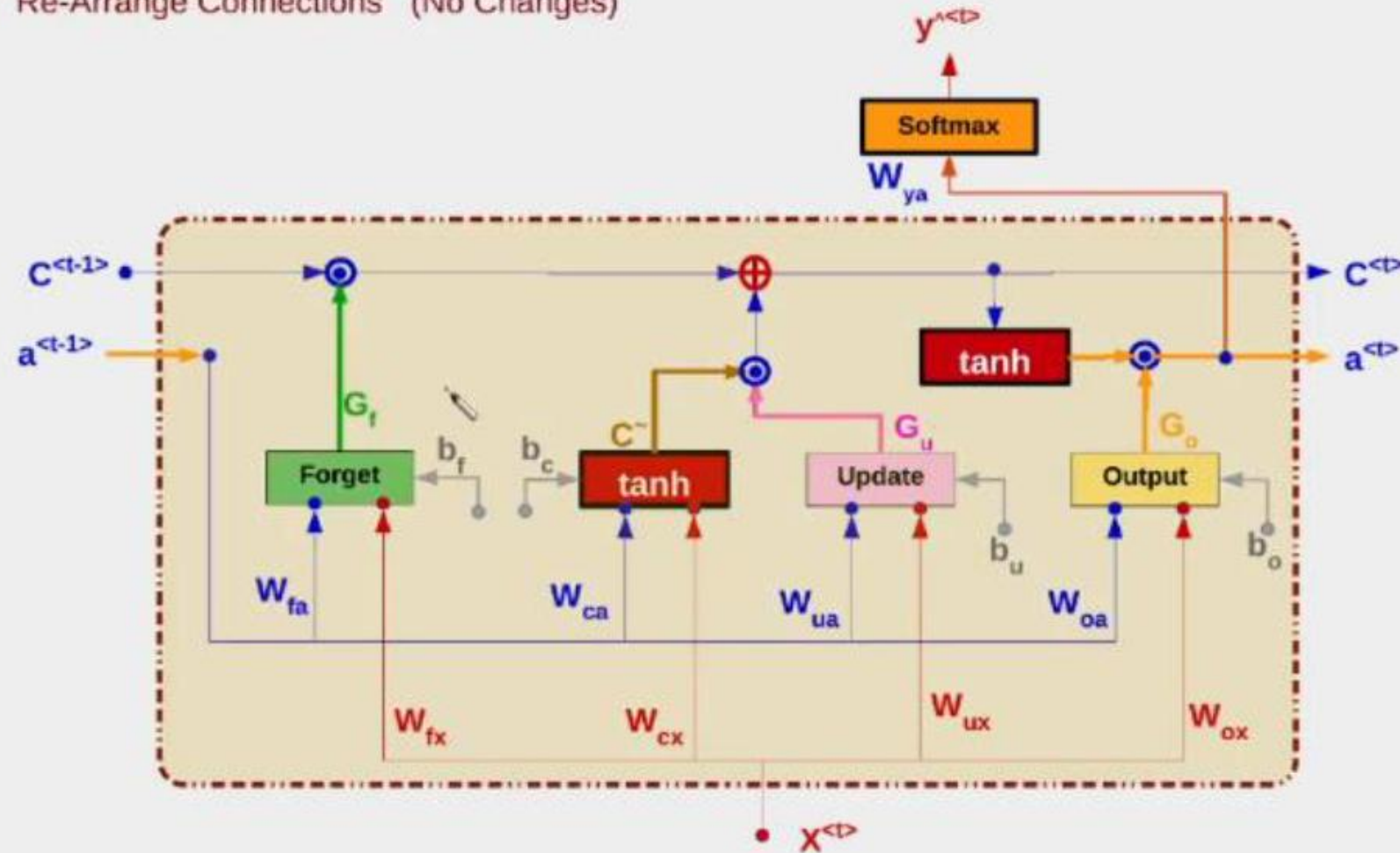
[5] Input to Control Gates is " $a^{<t-1>}$ "
[not " $C^{<t-1>}$ " as GRU]



Recurrent Neural Network

LSTM Unit

Re-Arrange Connections (No Changes)



Recurrent Neural Network

LSTM Unit

Gates

$$G_u = \text{Sigmoid}(W_{ua}a^{<t-1>} + W_{ux}x^{<t>} + b_u)$$

$$G_f = \text{Sigmoid}(W_{fa}a^{<t-1>} + W_{fx}x^{<t>} + b_f)$$

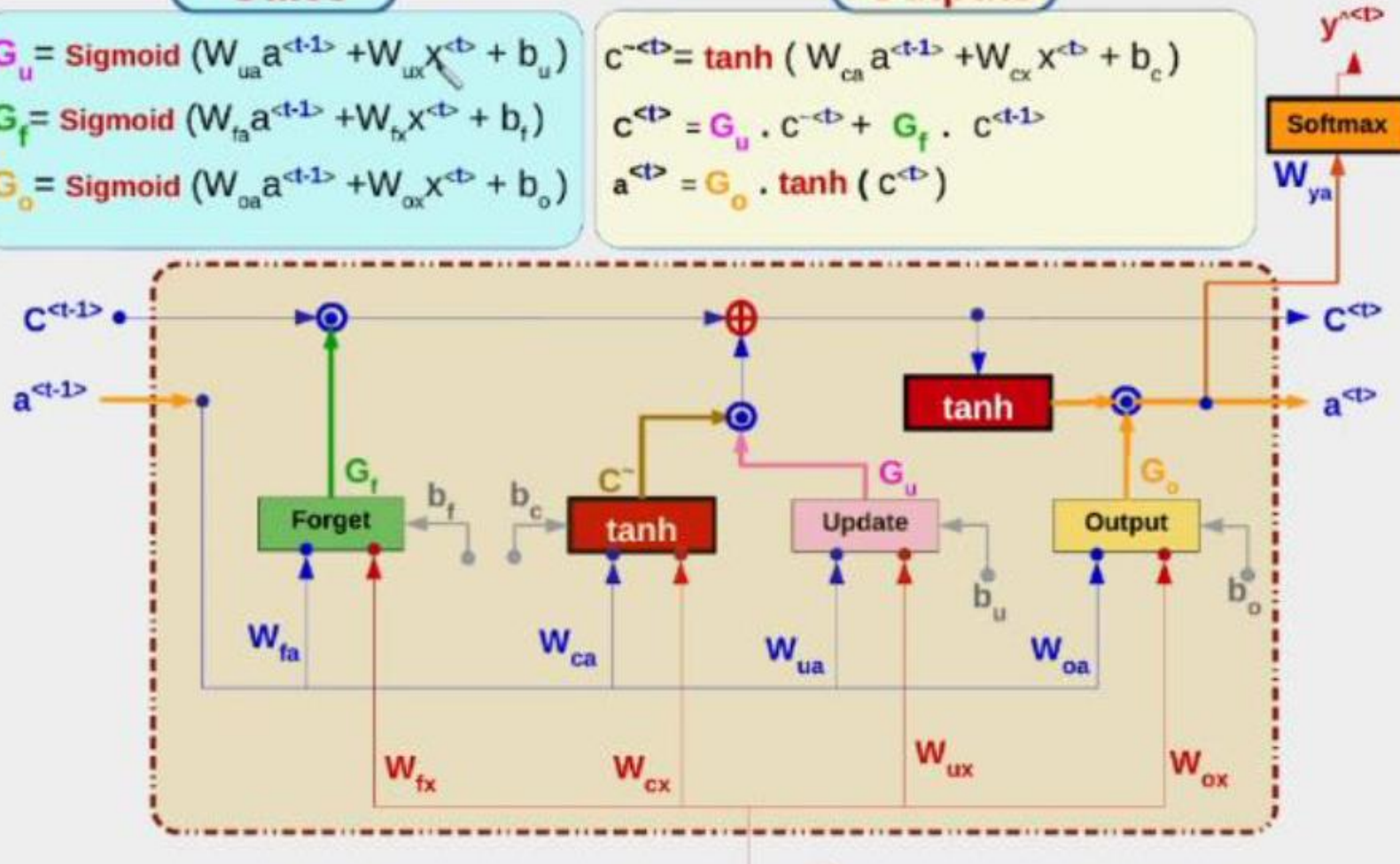
$$G_o = \text{Sigmoid}(W_{oa}a^{<t-1>} + W_{ox}x^{<t>} + b_o)$$

Outputs

$$c^{<t>} = \tanh(W_{ca}a^{<t-1>} + W_{cx}x^{<t>} + b_c)$$

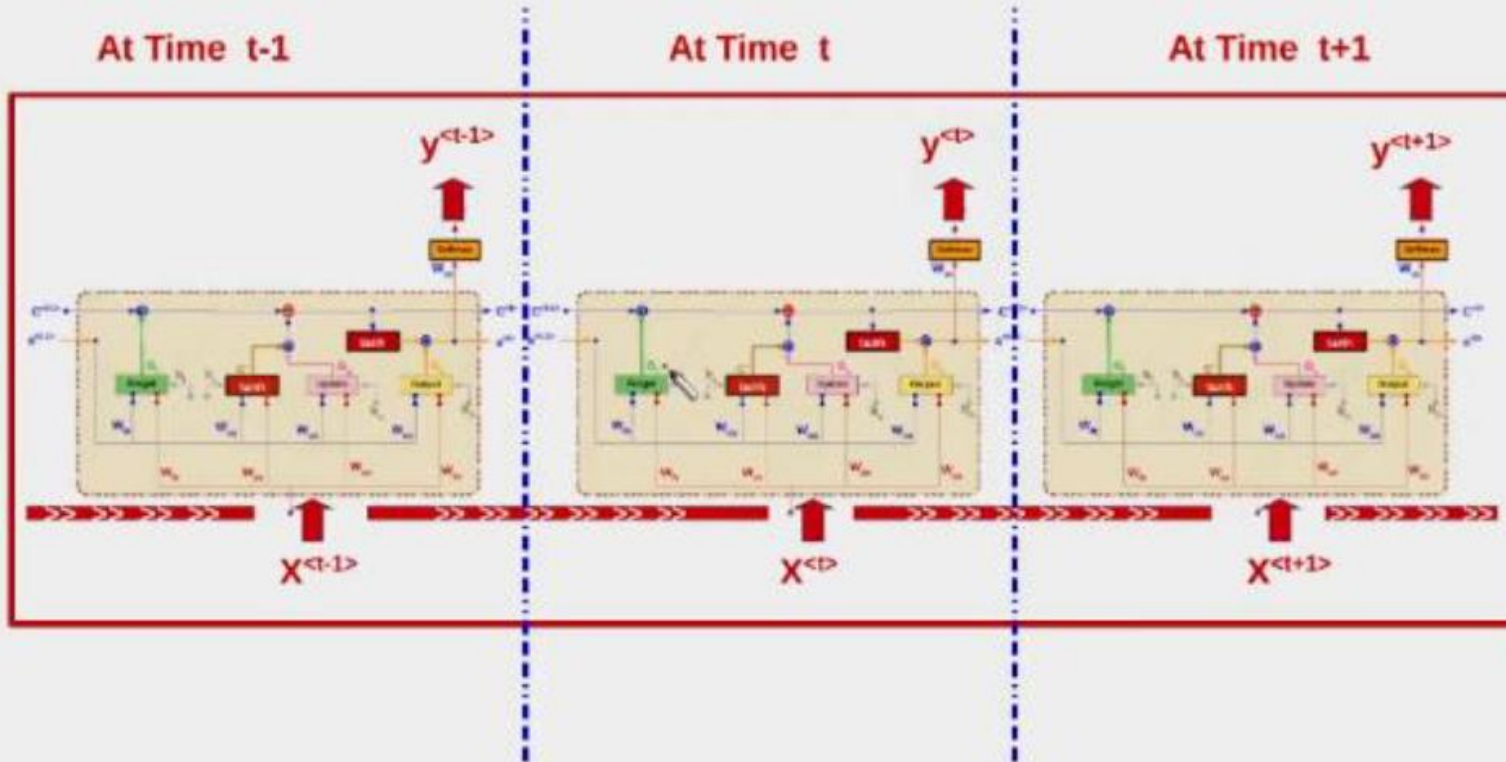
$$c^{<t>} = G_u \cdot c^{<t-1>} + G_f \cdot c^{<t-1>}$$

$$a^{<t>} = G_o \cdot \tanh(c^{<t>})$$



Recurrent Neural Network

Input Sequence to LSTM



Thank you